

Overview

 In this course, you will begin your journey to understand how to take the complexity out of testing graphical user interfaces and human-machine interfaces by getting to know Squish and how to test applications.

What Will I Learn ?

1. How do I manage Applications Under Test in the Squish IDE?
2. How do I create new tests in the Squish IDE?
3. How do I use verification points to validate GUI objects?

[Squish for Qt Tutorials | Squish 9.2](#)

[Installing Tesseract for Squish | Squish 9.2](#)

 A Beginners Guide to Software Testing.pdf

[GitHub - qt-learning/Basic-Squish-Usage · GitHub](#)

Managing AUTs

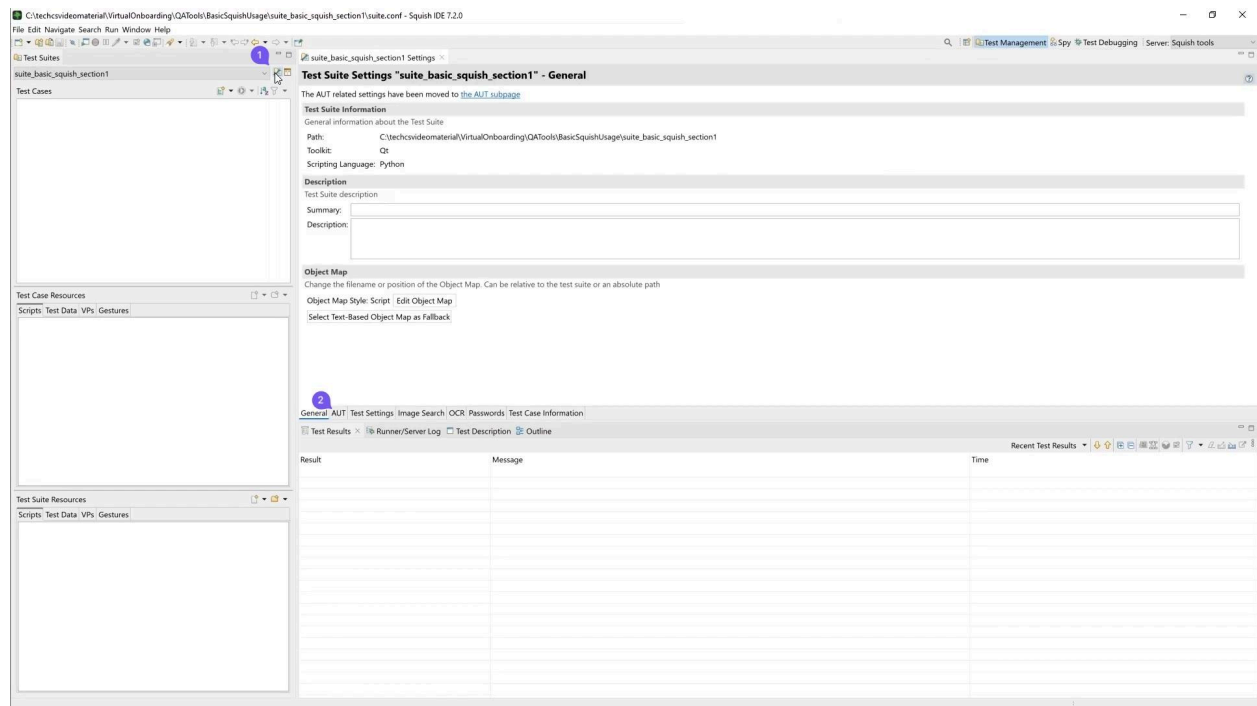
Before creating tests with Squish, it is important to ensure that the Squish IDE is properly configured to use the desired applications. All tests within Squish target an Application Under Test (AUT). Paths to AUTs can be stored with mappings that associate AUT paths to corresponding names used to specify which application is used in a test. AUT paths and mappings can be modified in the [Manage AUTs pane](#) by navigating to Edit > Server Settings... > Manage AUTs. Once an AUT path and corresponding name have been added via this menu, it is considered a Mapped AUT.

The Squish package comes with many built-in example applications that can be added to the list of Mapped AUTs. These examples can be found in the examples subdirectory within the Squish package directory: **SQUISHDIR/examples**.

i SQUISHDIR refers to the directory of your Squish installation

STEPS - How to configure AUTs in Squish IDE

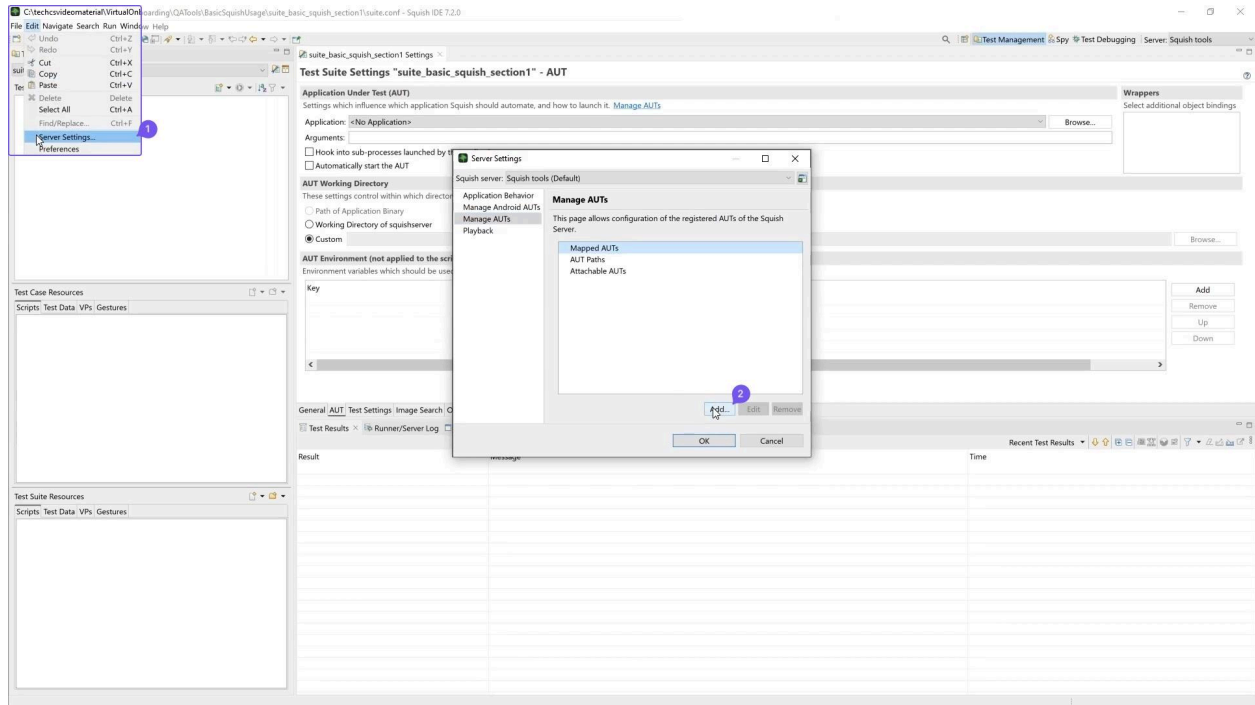
Test Suite Settings



Select the "Test Suite Settings" icon in the Test Suites view, then go to the AUT tab.

Notice under "Application Under Test" that no default AUT is set.

Managing AUTs



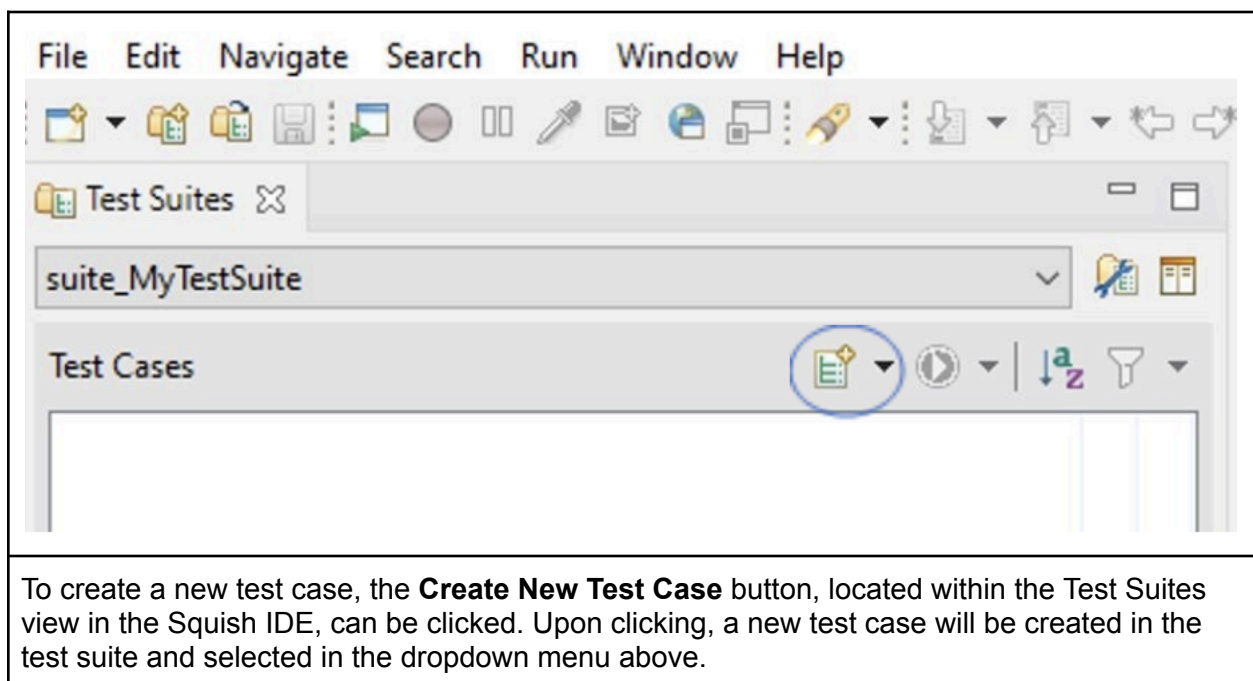
Go to "Edit > Server Settings... > Manage AUTs". In "Manage AUTs" you can view, add, edit, or remove Mapped AUTs. Choose "Mapped AUTs" and click "Add" to open a dialog for selecting an application executable. Select the addressbook example application from the Squish package's examples directory.

Selecting the AUT

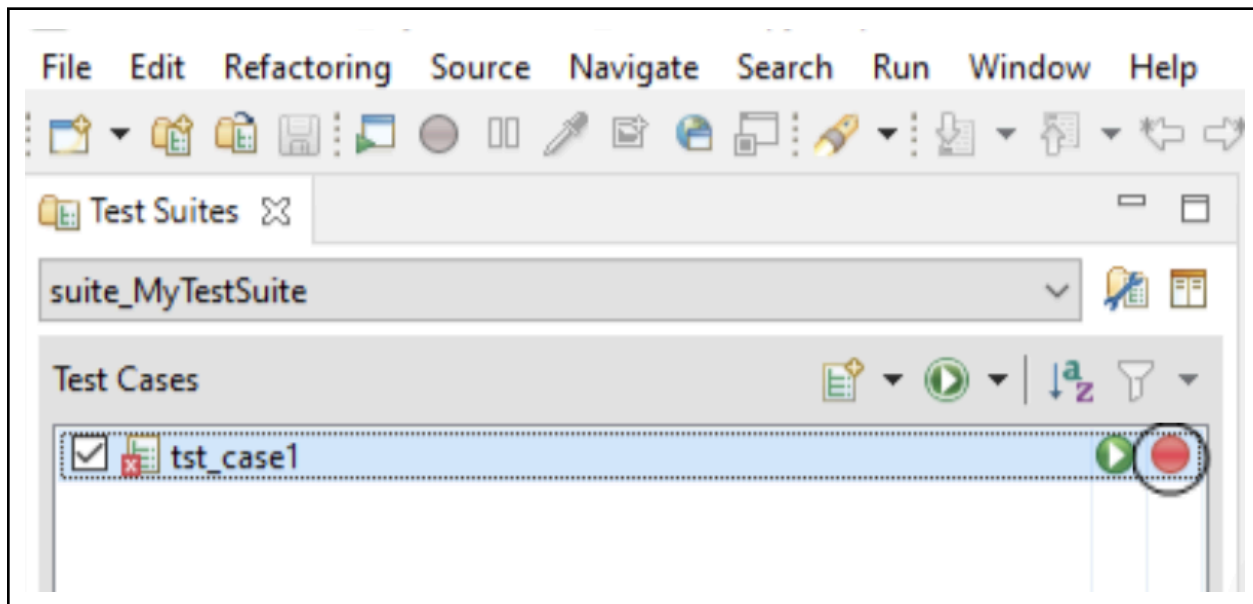
Creating Test Suites and Test Cases

A test suite is a collection of individual test cases configured to act on the same AUT by default. Multiple test cases can be run in succession in a single test suite. A test suite can be created in the Squish IDE by going to **File > New Test Suite**. This will prompt a [New Squish Test Suite](#) wizard where the project settings are defined.

Once a test suite has been created, it is possible to create test cases in that test suite. A test case contains at least one test script and should be used to test individual application functions. It is important to keep test cases as simple as possible, so it is easier to diagnose problems when a test case fails.



Recording a Test

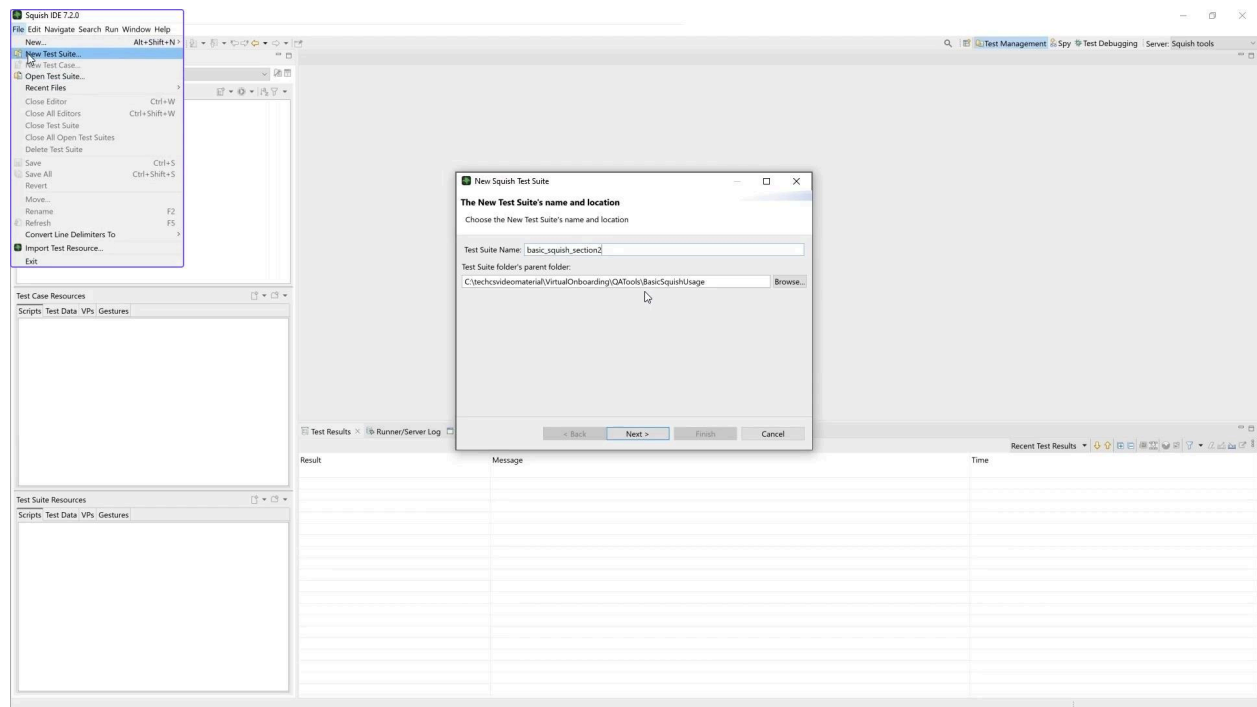


It is possible to write test scripts manually by utilizing methods from the Squish API, or to generate scripts automatically by using the record feature within the Squish IDE. By clicking on the **Record Test** button next to the name of the selected test case, Squish can record GUI activity and generate a script based on that activity.

The Record Test feature will open both the AUT and a [Control Bar window](#). All activities performed on the AUT will be recorded and used to generate a script when the Stop Recording button within the Control Bar window is clicked. The Control Bar window allows for the insertion of verification points into the script and stops recording when a test is complete.

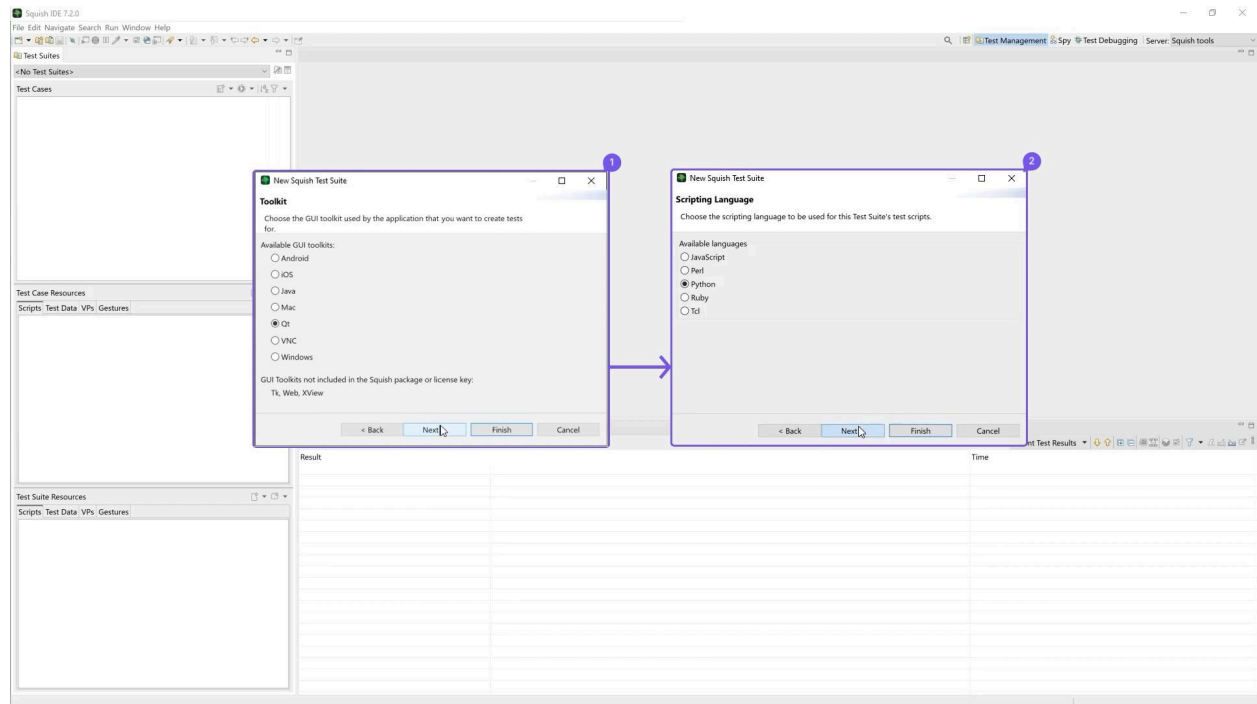
STEPS - create a new test and use the record feature within the IDE

01 Creating a Test Suite



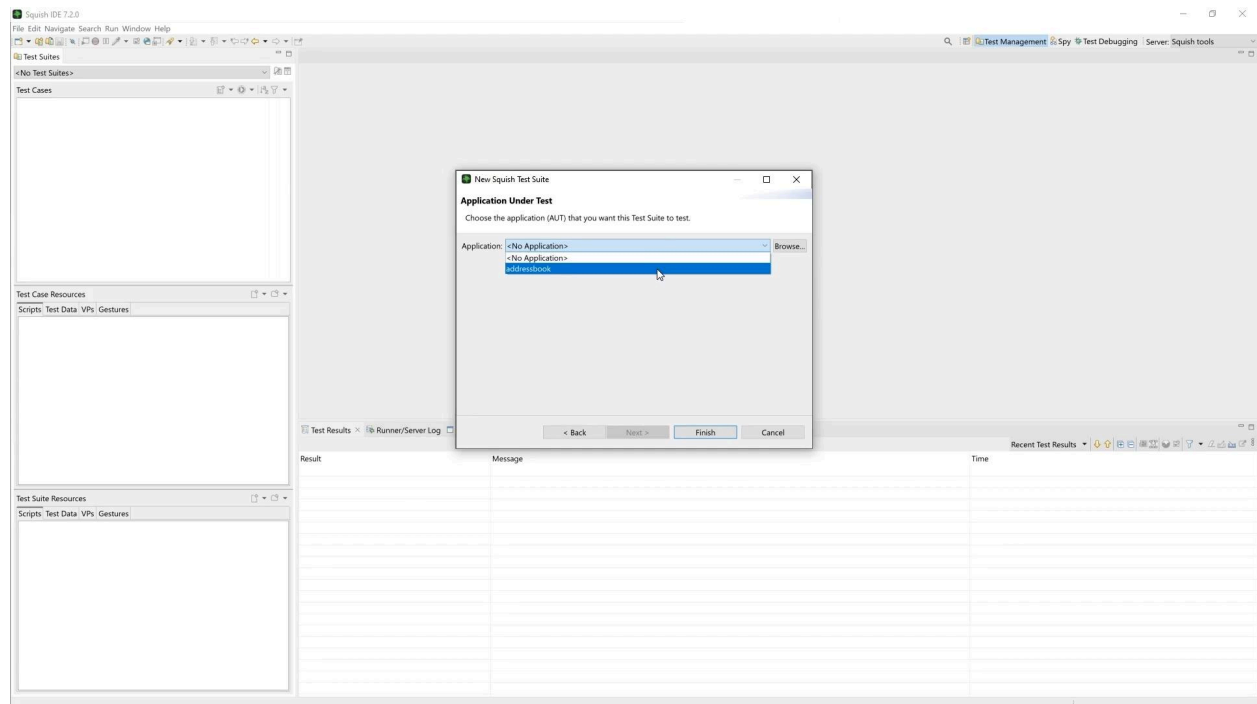
Select "File > New Test Suite" to start the New Squish Test Suite wizard. Enter a name and parent folder to save the test suite.

02 Test Suite Settings



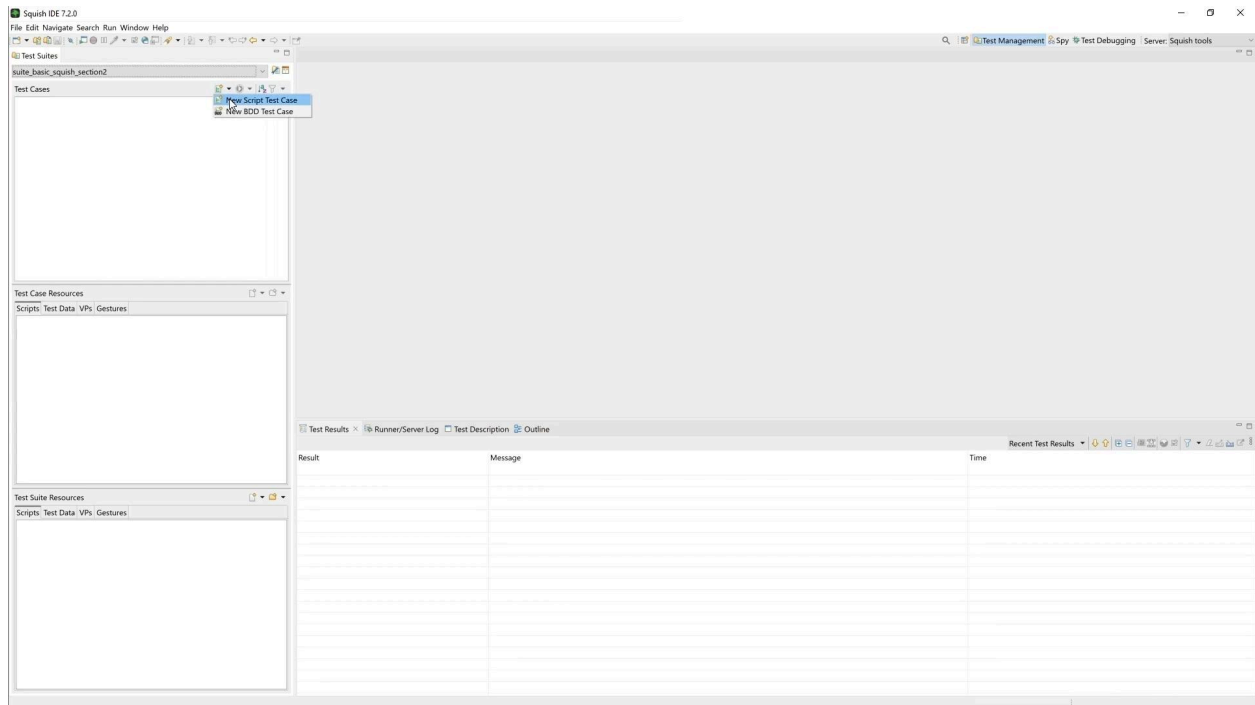
Choose a GUI toolkit that matches the Squish package and the toolkit used for the AUT, in this case Qt. Select a scripting language for test cases, in this case Python.

03 Selecting the AUT



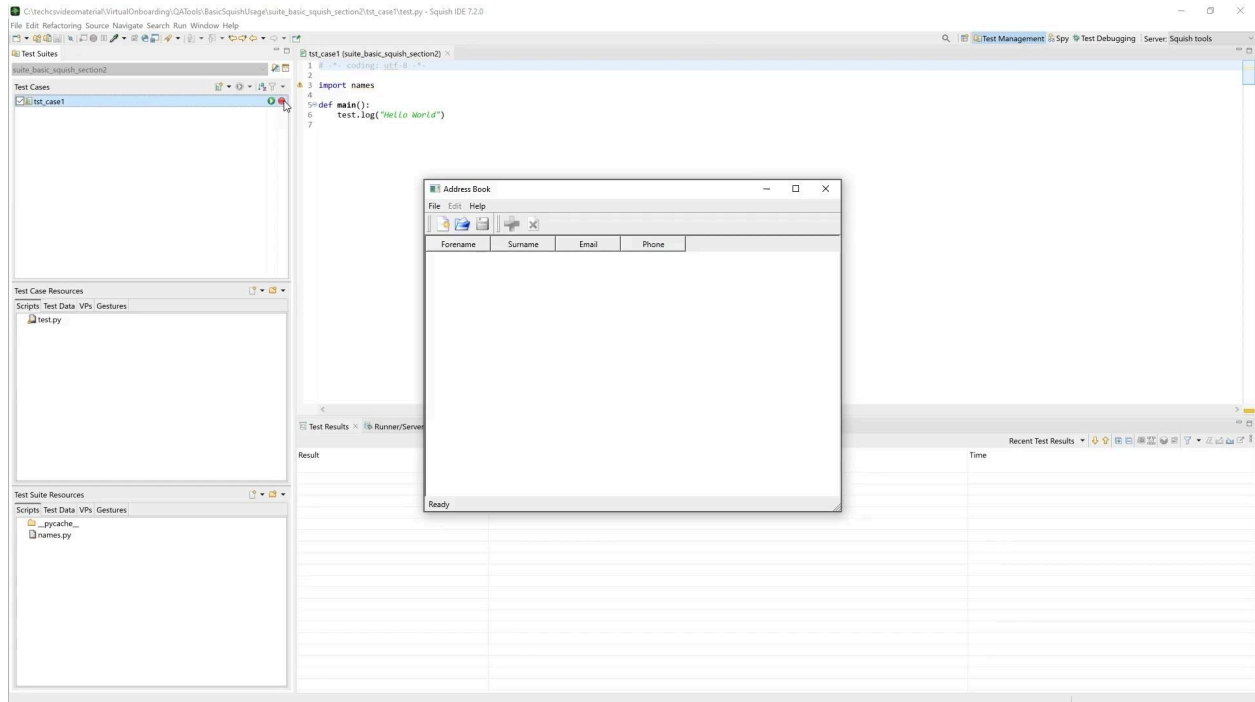
Select the application (AUT) you want the test suite to test.

04 Creating a Test Case



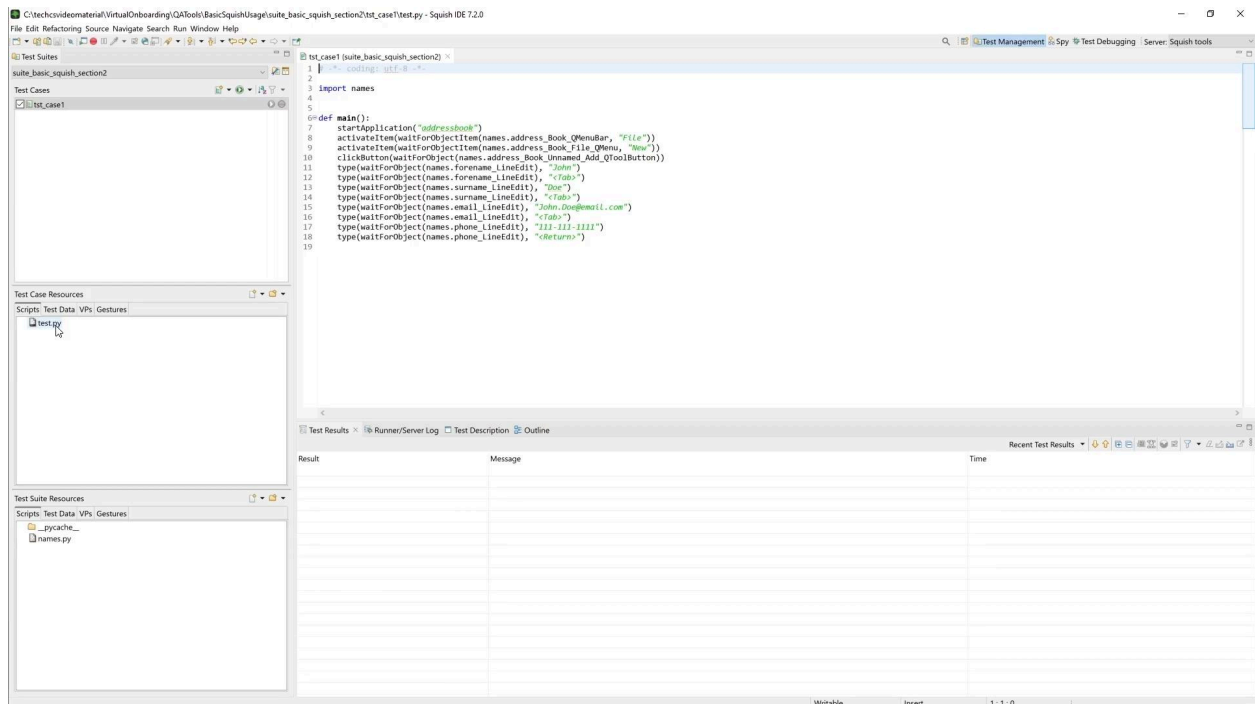
Click the Create New Test Case button, located within the Test Suites view in the Squish IDE

05 Recording a test



Click the record button next to the test case to start recording interactions with the AUT, this will open the application and the control bar. Perform actions you would like to test.

06 Stop Recording



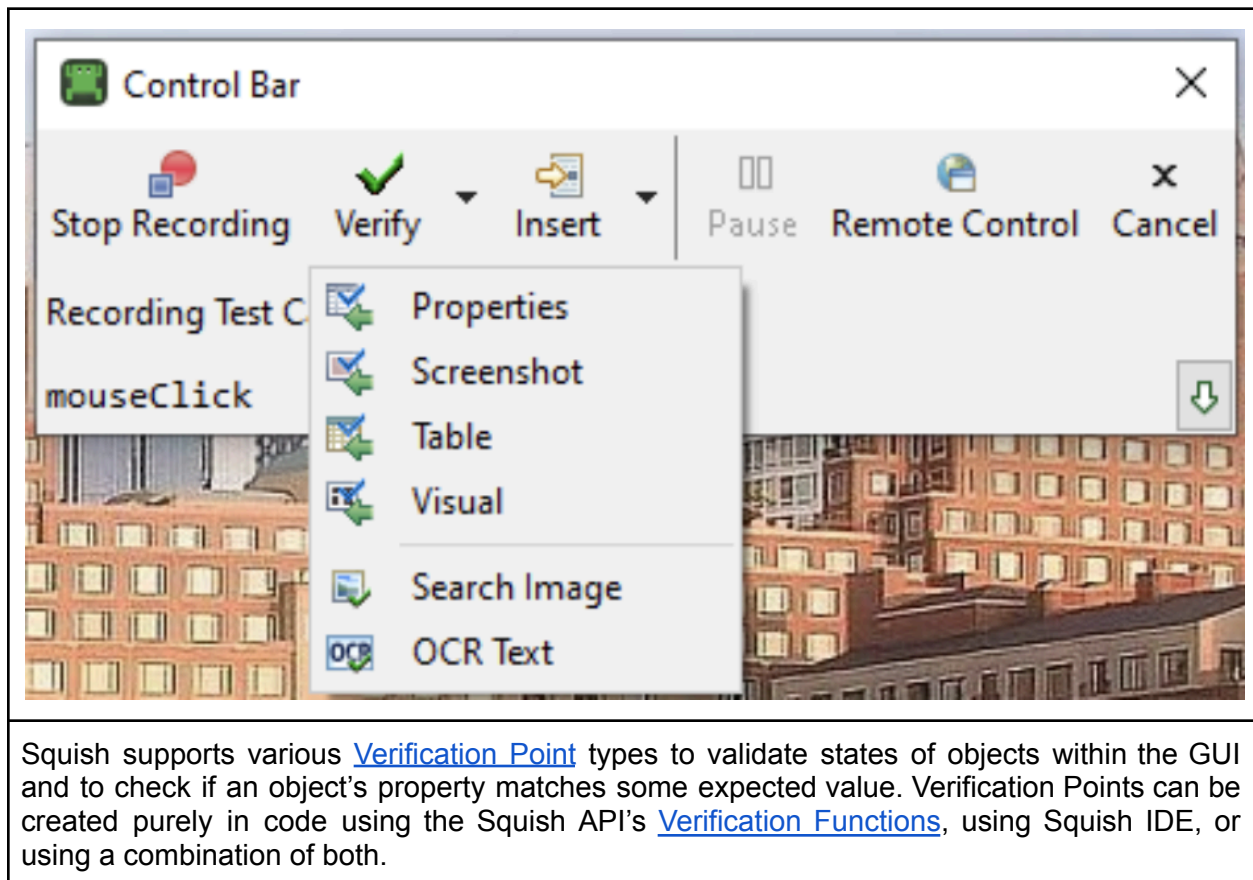
When you stop recording, a script will be generated based on the recorded actions.
These will be found in test.py.

07 Running a test



Use the play button to execute the recorded test, replicating the recorded actions.

Verification Types



Squish supports various [Verification Point](#) types to validate states of objects within the GUI and to check if an object's property matches some expected value. Verification Points can be created purely in code using the Squish API's [Verification Functions](#), using Squish IDE, or using a combination of both.

To create a verification point using the point-and-click interface, first select a verification type from the Verify dropdown menu in the [Control Bar window](#). This will prompt the [Verification Point Creator view](#) and allow for the creation of a verification point based on the type of verification selected. The types of verifications available are:

- Properties
 - Verifies a GUI object based on an object property and compares it to an expected value.
- Screenshot
 - Verifies GUI by taking a screenshot of a selected area and confirming that it matches an expected image.
- Table
 - Verifies the contents of a table to an expected table.

- Visual
 - Verifies GUI by selecting a visual section of the UI.
- Search Image
 - Verifies that an image is present somewhere in the window where the AUT is running.
- OCR Text
 - Verifies that some text is present within a defined region.

 **During recording, Verification Points can be saved and inserted into a test script by clicking Save and Insert Verification in the Verification Point Creator view.**

Property Verification

Property verification verifies a GUI object based on an object property and compares it to an expected value. For example, it is possible to compare the text value of a GUI object with some expected string value.

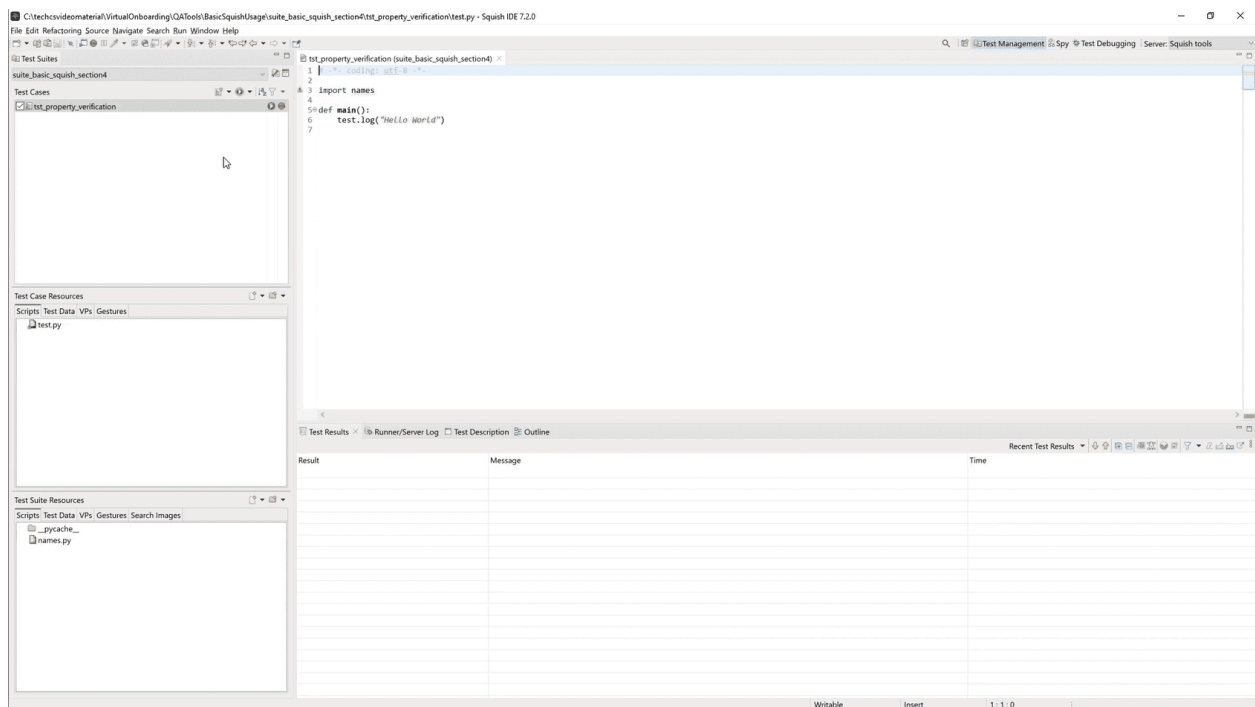
Property verifications are created by default as **Scriptified Properties Verification Points**. **Scriptified Properties Verification Points** are formatted as code, so it is easy to read, modify, or delete them. It is also possible to create Properties Verification Points which save a property value to be used in the verification as a resource. This property is saved by default in **Test Suites > Test Case Resources > VPs**. **Scriptified Properties** will use the [test.compare\(\)](#) function to compare a GUI element's property value to some expected value. A **Properties Verification Point** will use the [test.vp\(\)](#) to execute the verification point.

Python

```
test.compare(str(waitForObjectExists(names.o56_Text).text), "56")
```

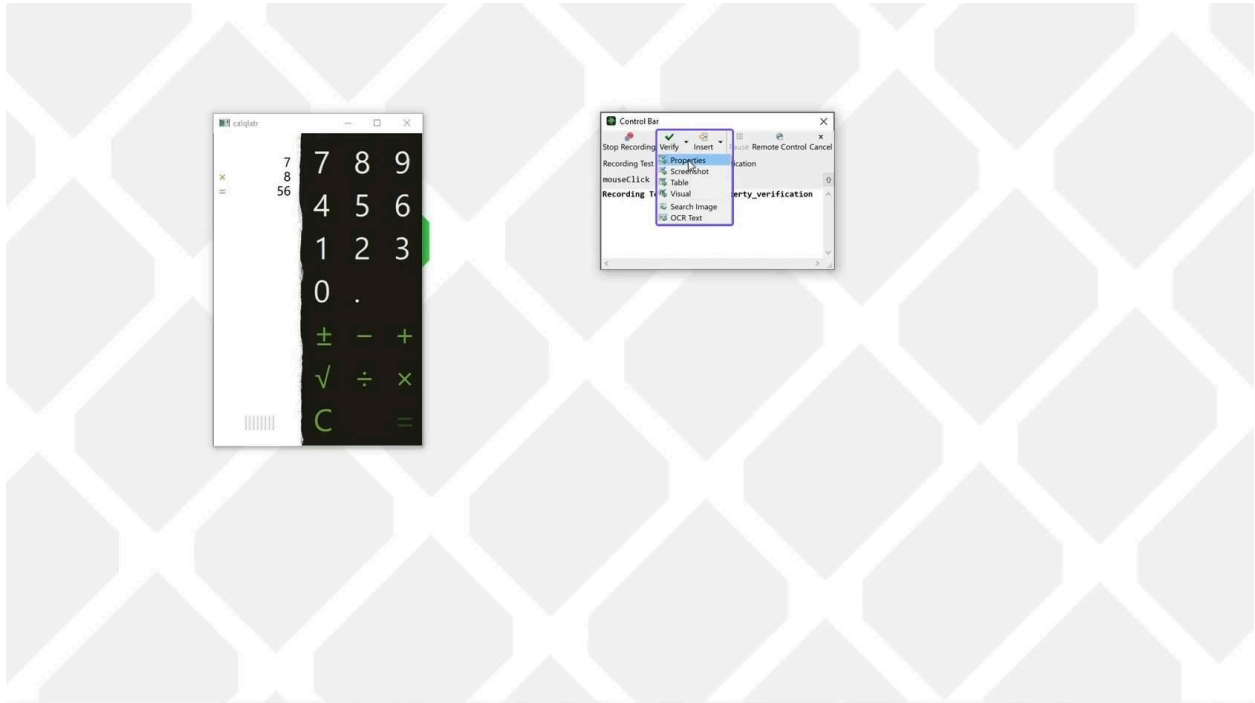
STEPS - How to Create Property Verifications

01 Creating a new test case



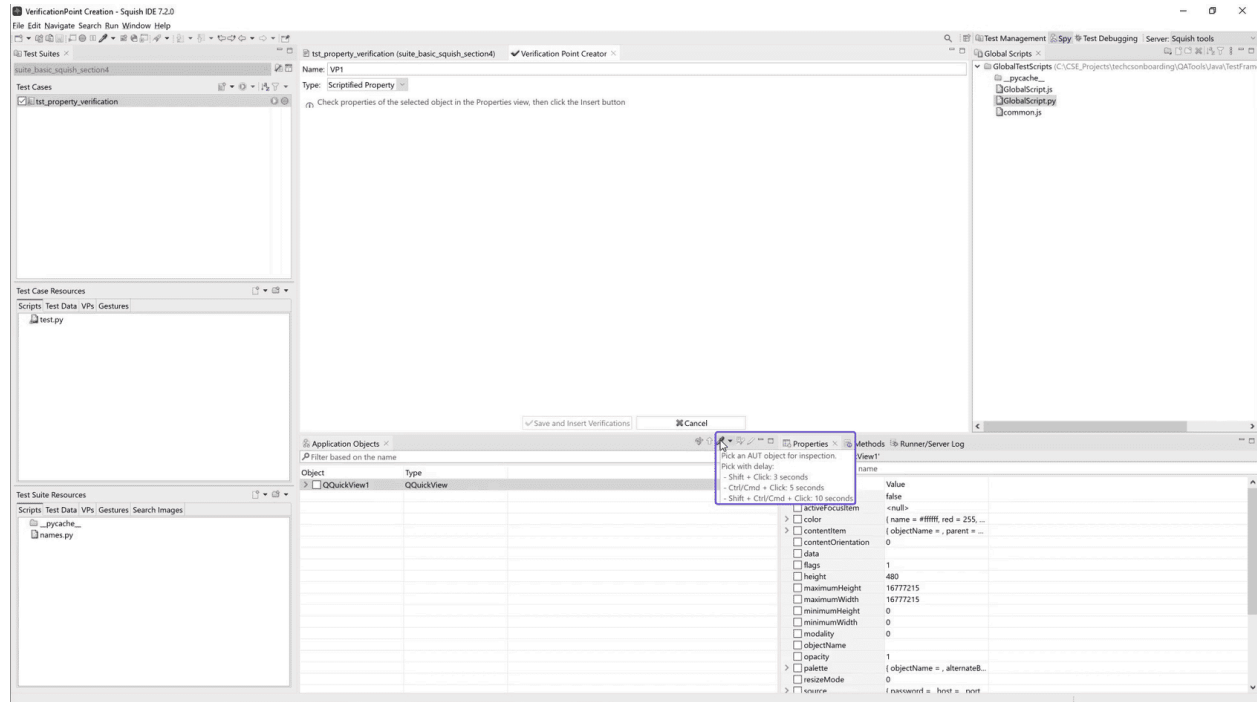
Click "New Script Test Case" button in Test Suites view. Rename the test case to "tst_property_verification."

02 Recording a test and initiating property verification.



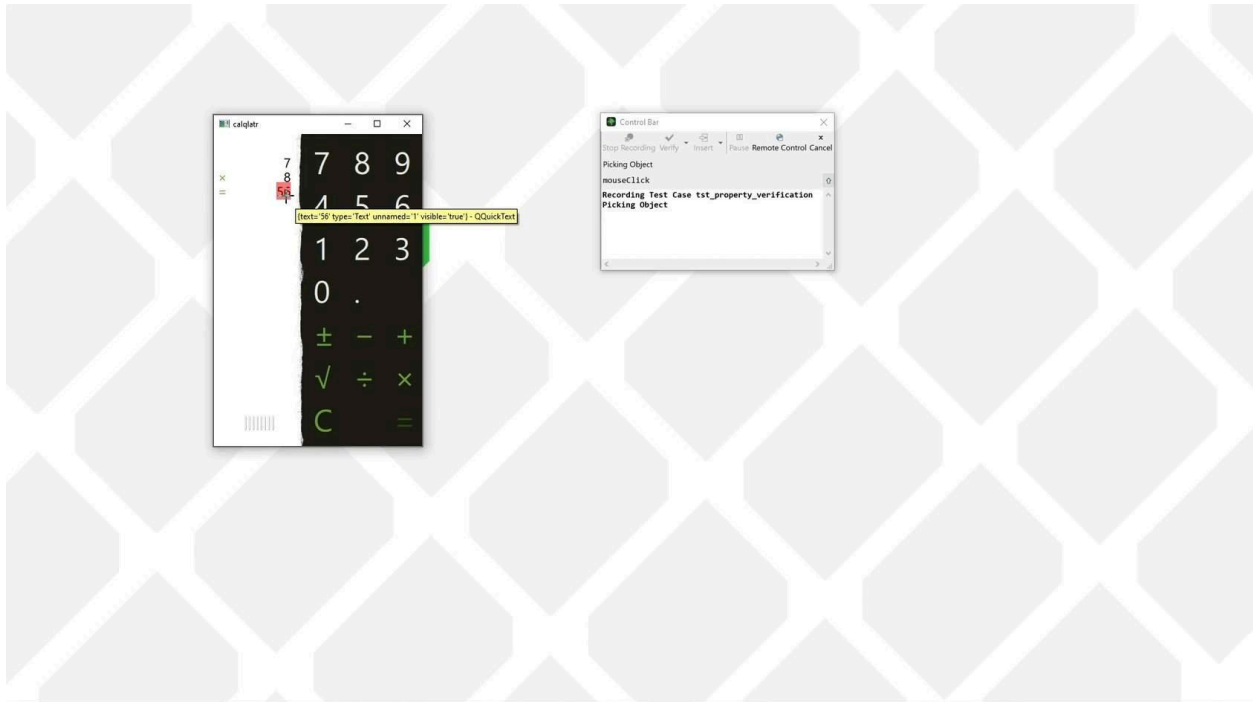
Start recording by selecting the record button next to the test case. Perform a mathematical operation on the calculator AUT. Now there is a result on the calculator, we can test this result. In the control bar, use the dropdown next to the Verify button to select property verification, causing the Squish IDE to reappear.

03 Selecting the picker tool



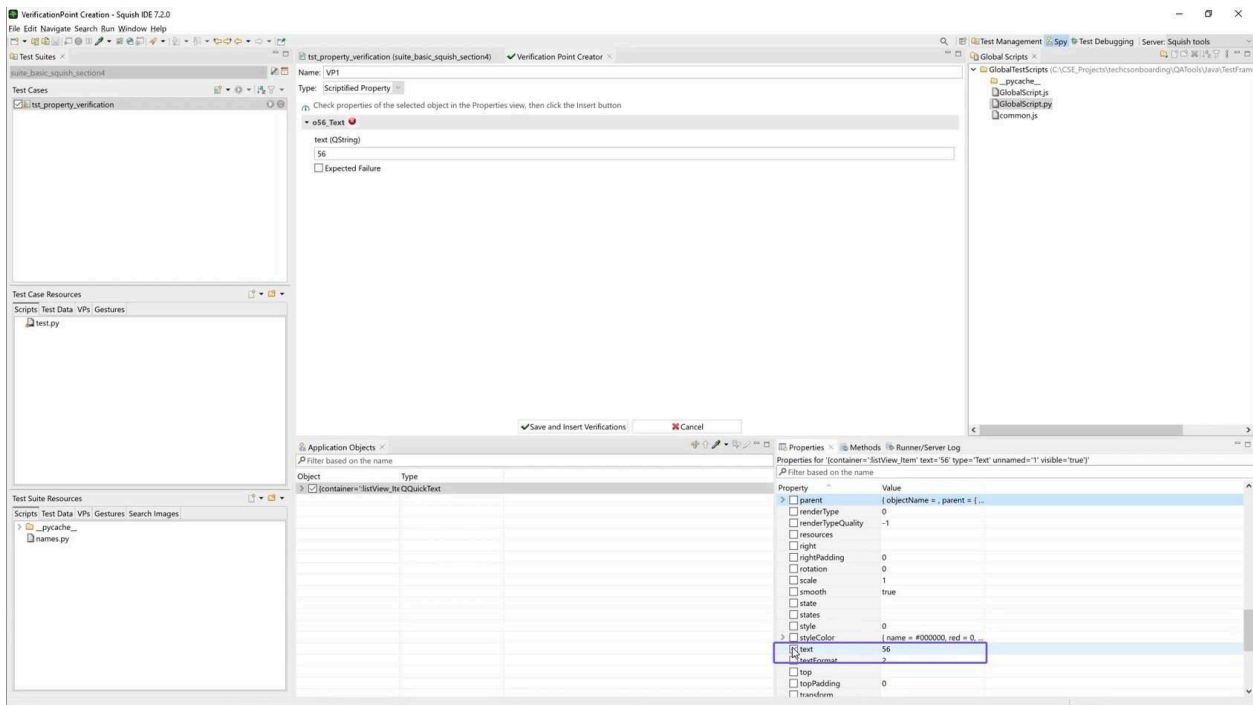
We have now entered into the Spy view, providing a hierarchical tree of the application elements. We can use the picker tool to interact with the AUT. Select the tool and we will be returned to the AUT.

04 Selecting the object for verification



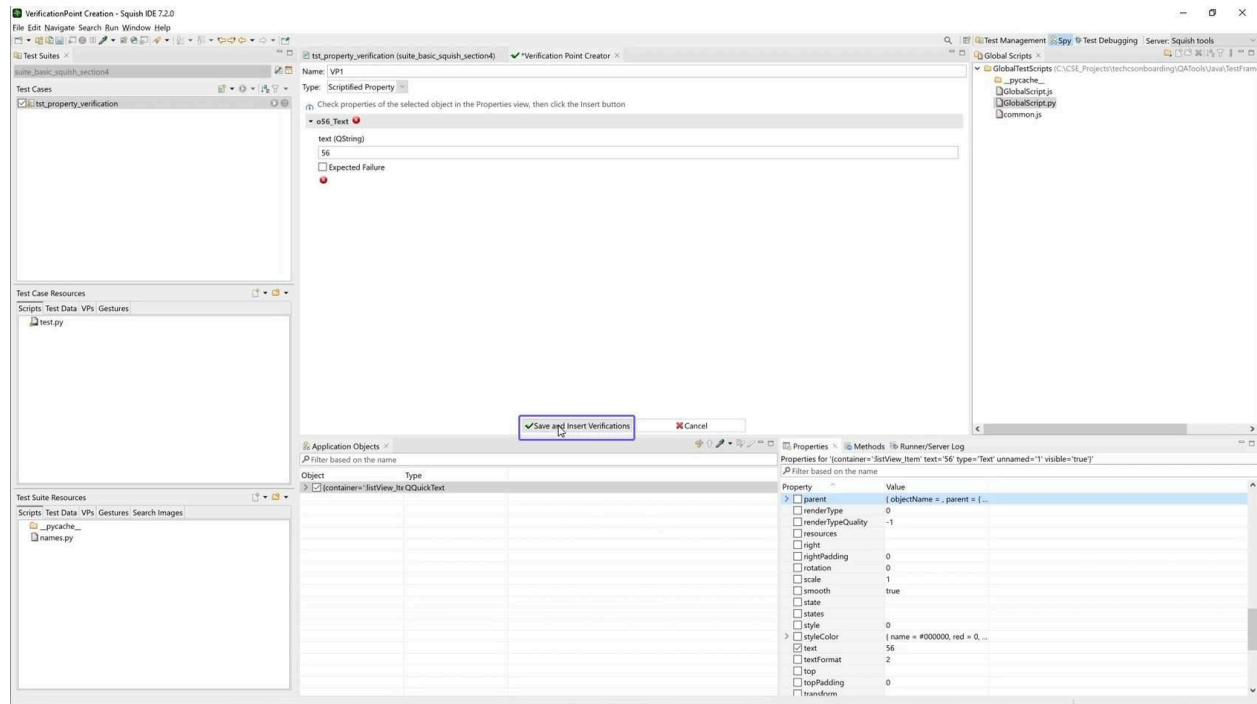
Select the QQuickText object displaying the product “56” in the calculator.

05 Choosing the property



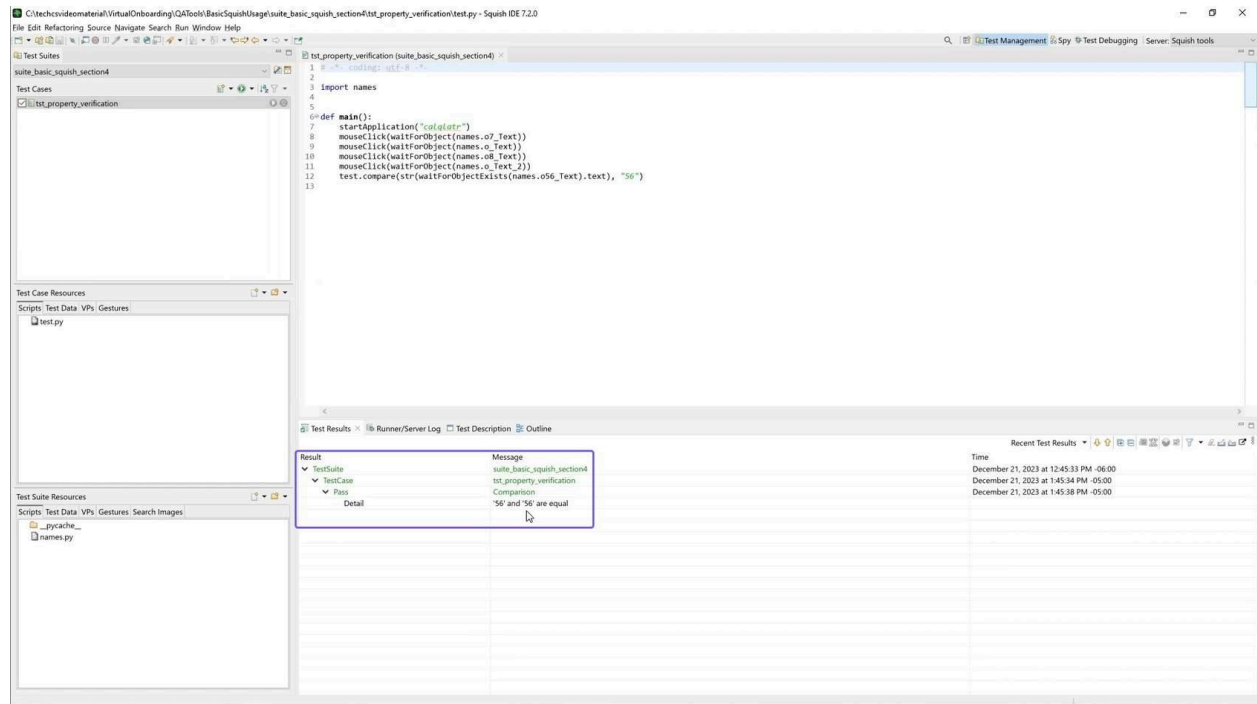
In the Application Objects view, select the text property of the chosen object for verification.

06 Saving the verification point



Use the Verification Point Creator to save the property verification and resume recording. Return to the AUT and finish the recording by clicking "Stop Recording" in the Control Bar.

07 Run and verify



Run the test using the run button next to the test case. Observe the multiplication operation and the test passing in the Test Results view. Expand the details in the Test Results view to see the comparison of expected value "56" with the actual value.

Screenshot Verification

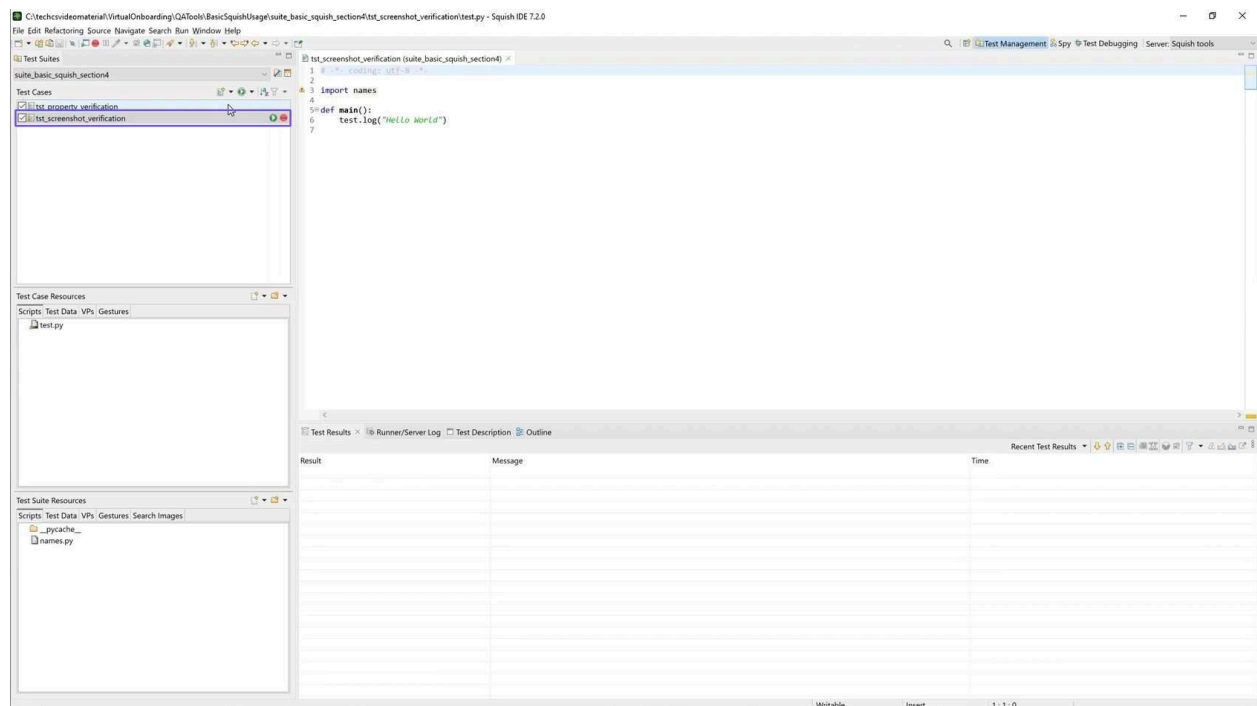
Screenshot verification takes a screenshot of a selected area of the GUI and confirms that it matches an expected image. A Screenshot verification point is used to make a pixel-by-pixel comparison within a test script with the [test.vp\(\)](#).

Python

```
test.vp("VP1")
```

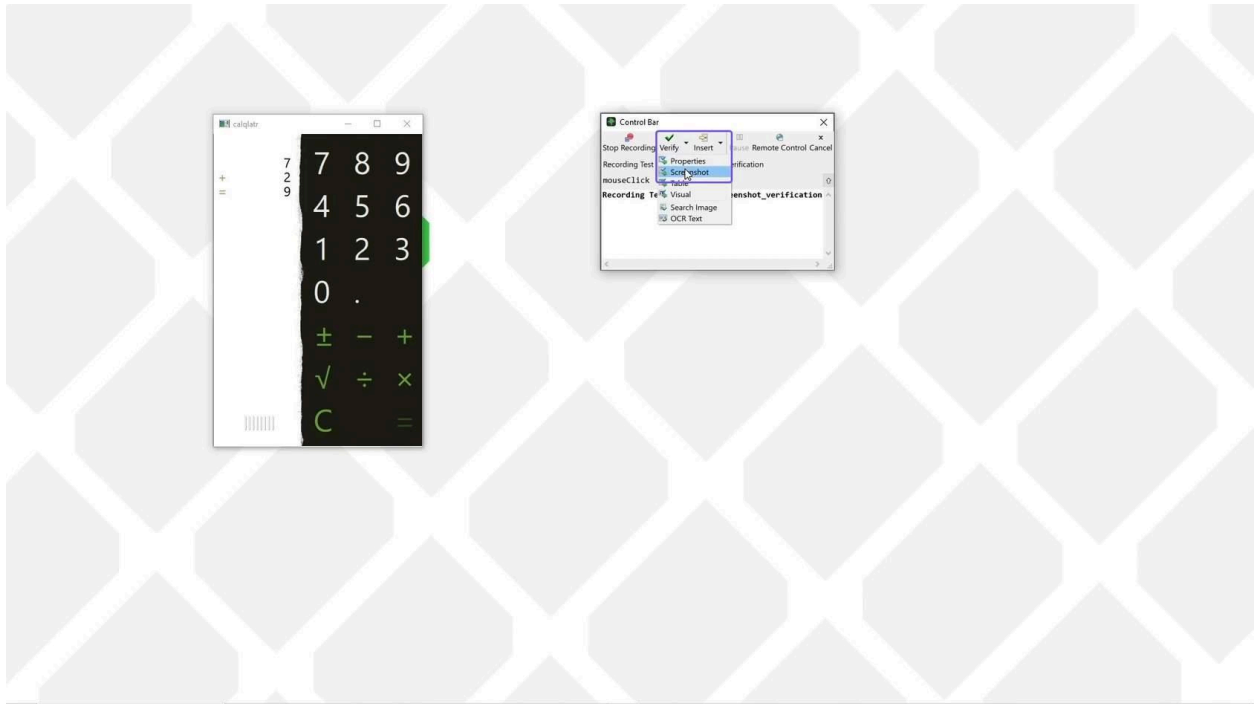
STEPS

01 Creating a new test case



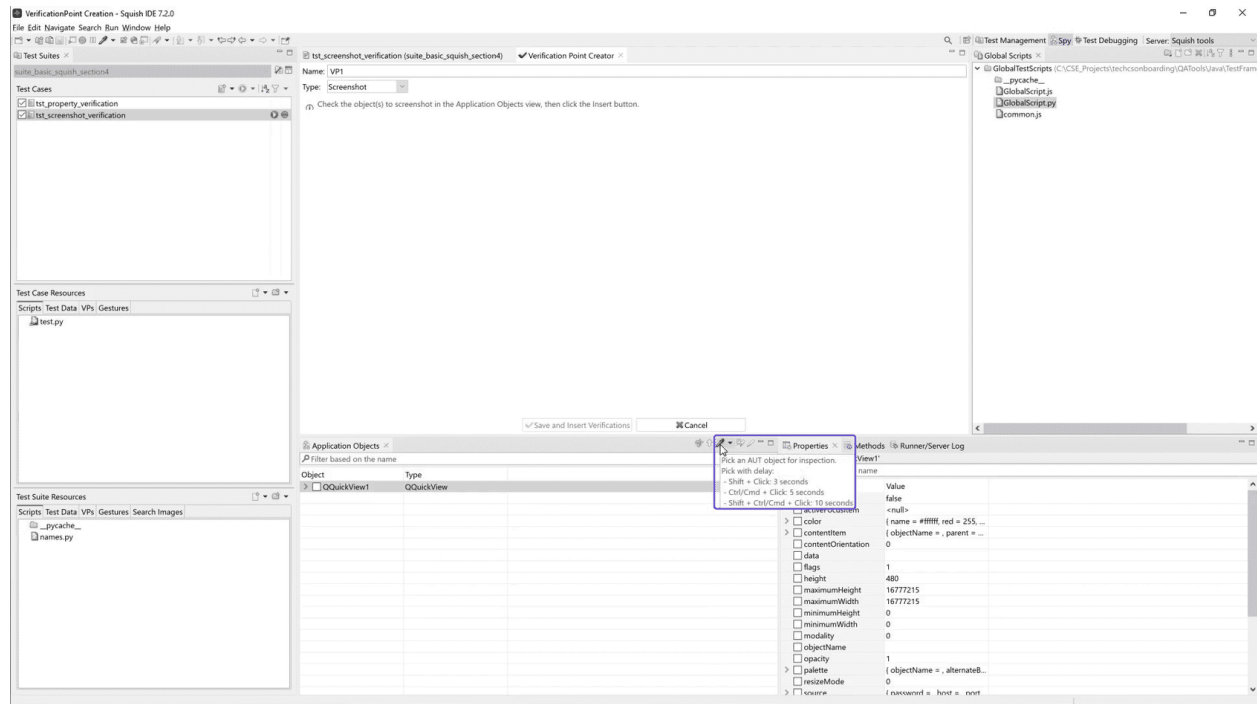
Add a new test case, “tst_screenshot_verification,” to the existing test suite using the calculator example as the AUT.

02 Recording a test and initiating screenshot verification.



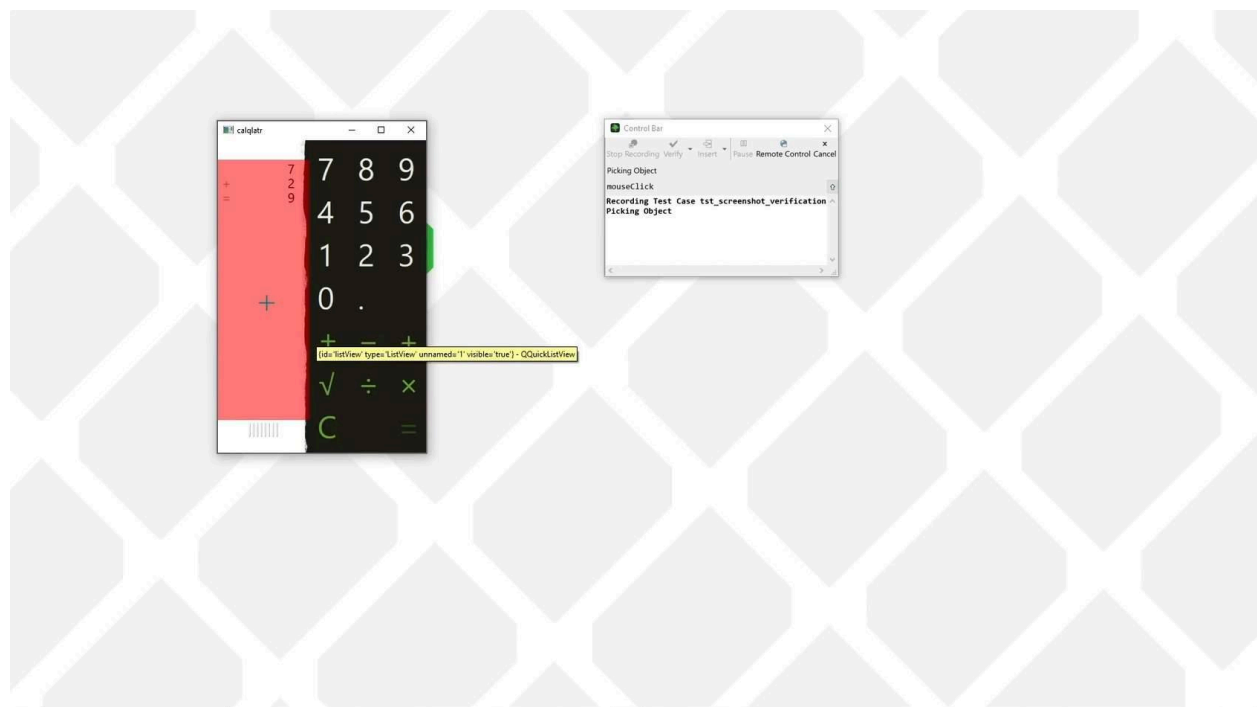
Click the record button next to the test case. Perform a simple operation on the calculator. In the control bar, select the dropdown next to the Verify button and choose screenshot verification, causing the Squish IDE to reappear.

03 Selecting the picker tool



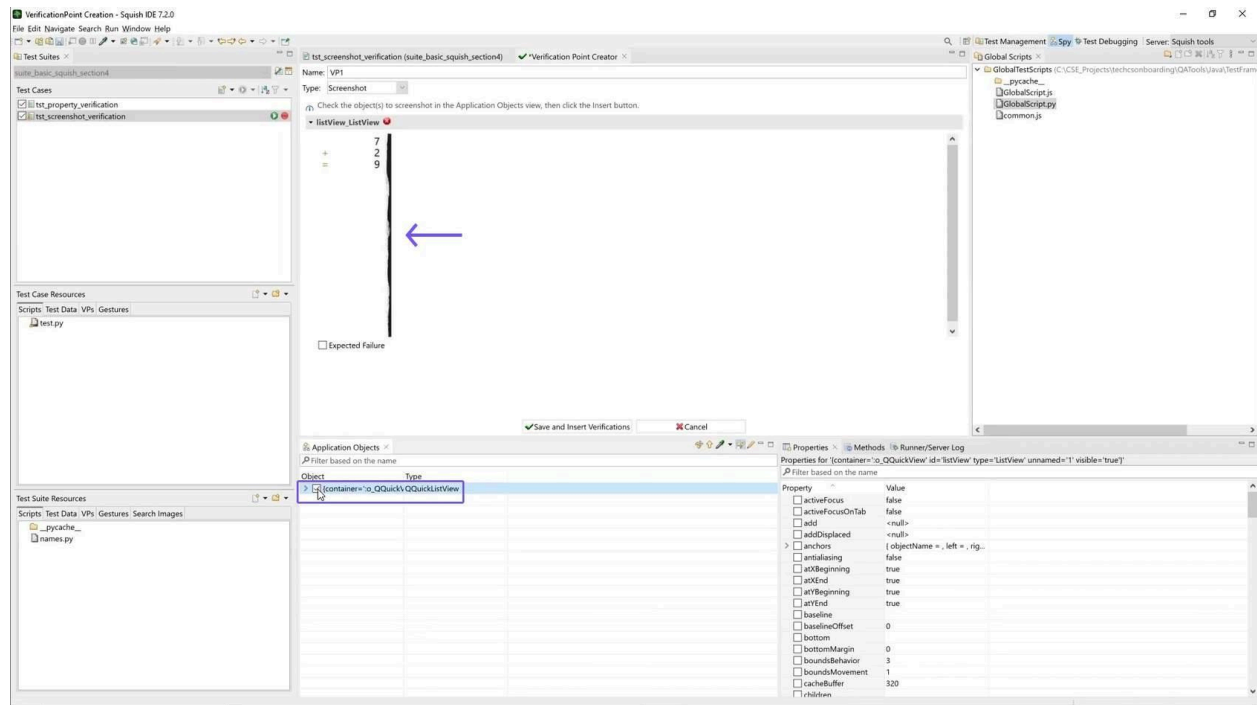
Select the picker tool and we will be returned to the AUT.

04 Selecting the object for verification



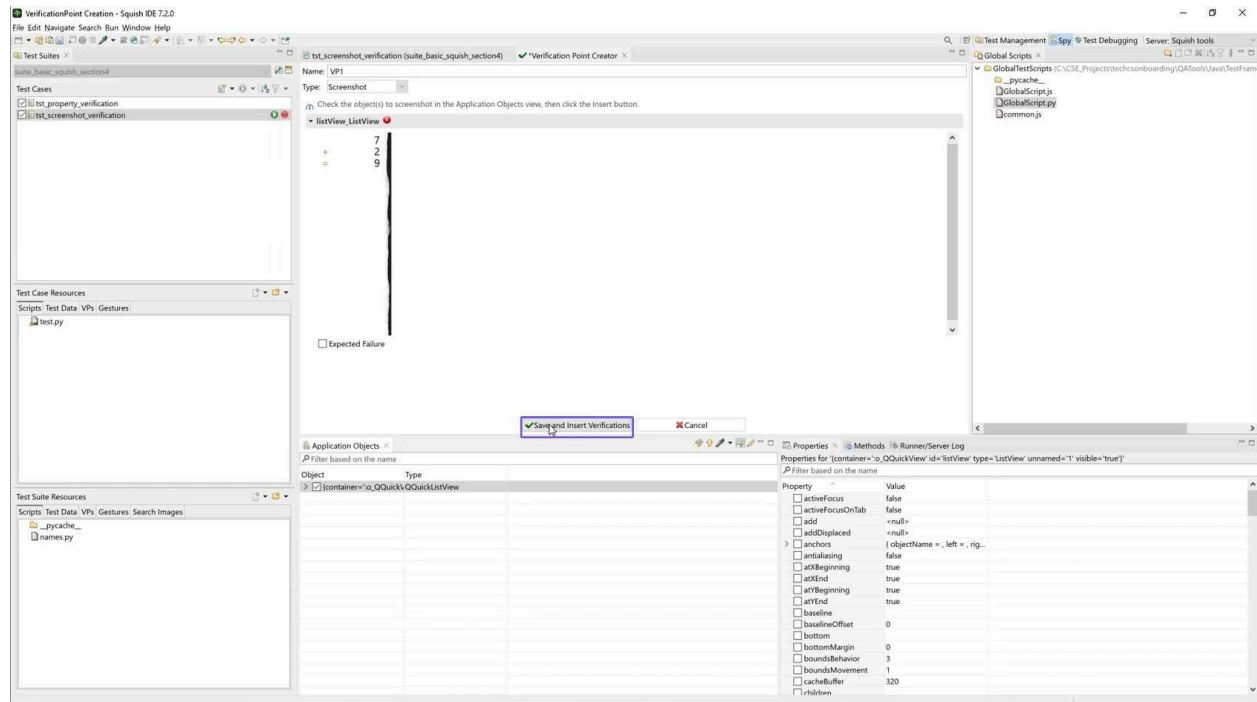
Use the Picker tool to select the QQuickListView object containing the calculator results from the AUT.

05 Choose the object in the application object view



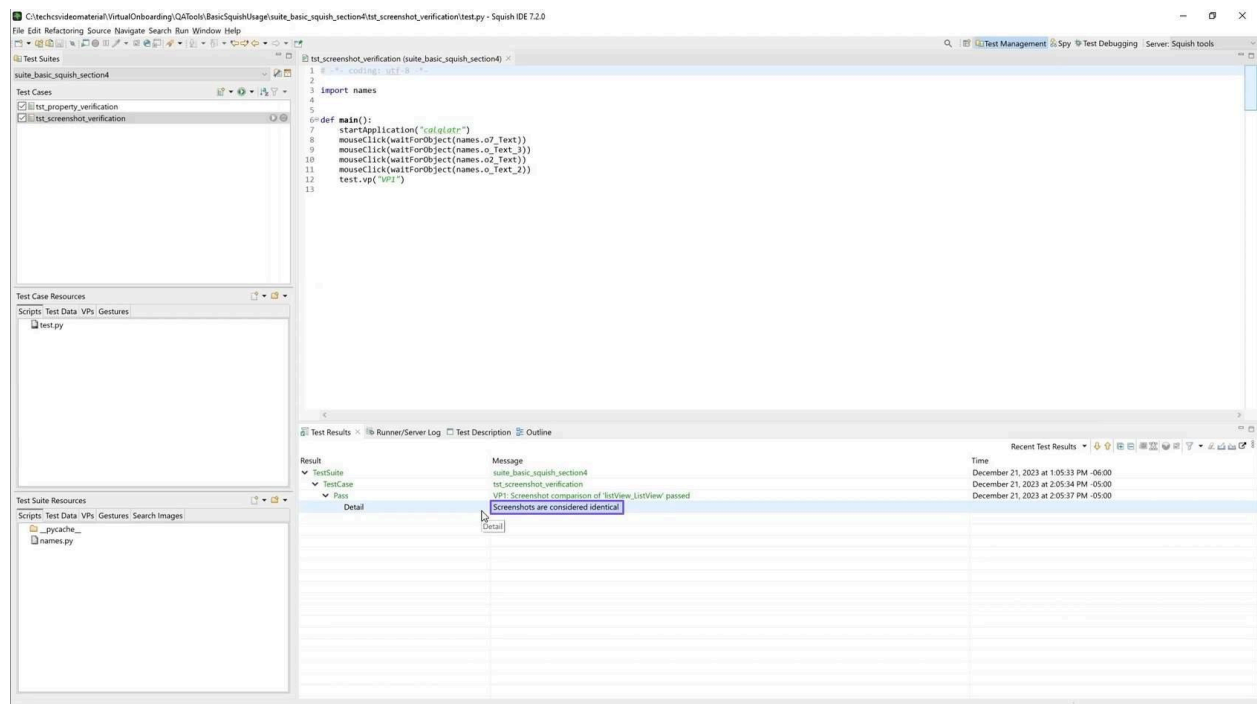
Select the object in the Application Objects view. A screenshot of the object appears in the Verification Point Creator view.

06 Saving the verification point



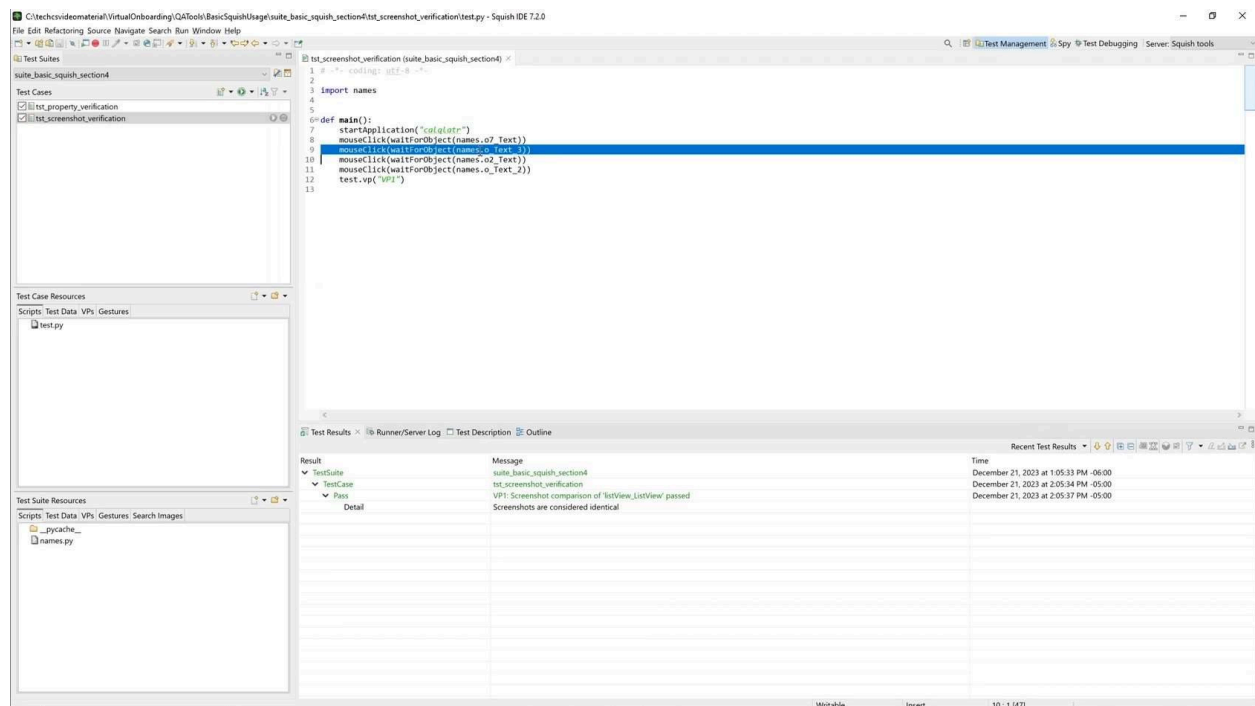
Click "Save and Insert Verifications" to create the screenshot verification point and resume recording.

07 Run and verify



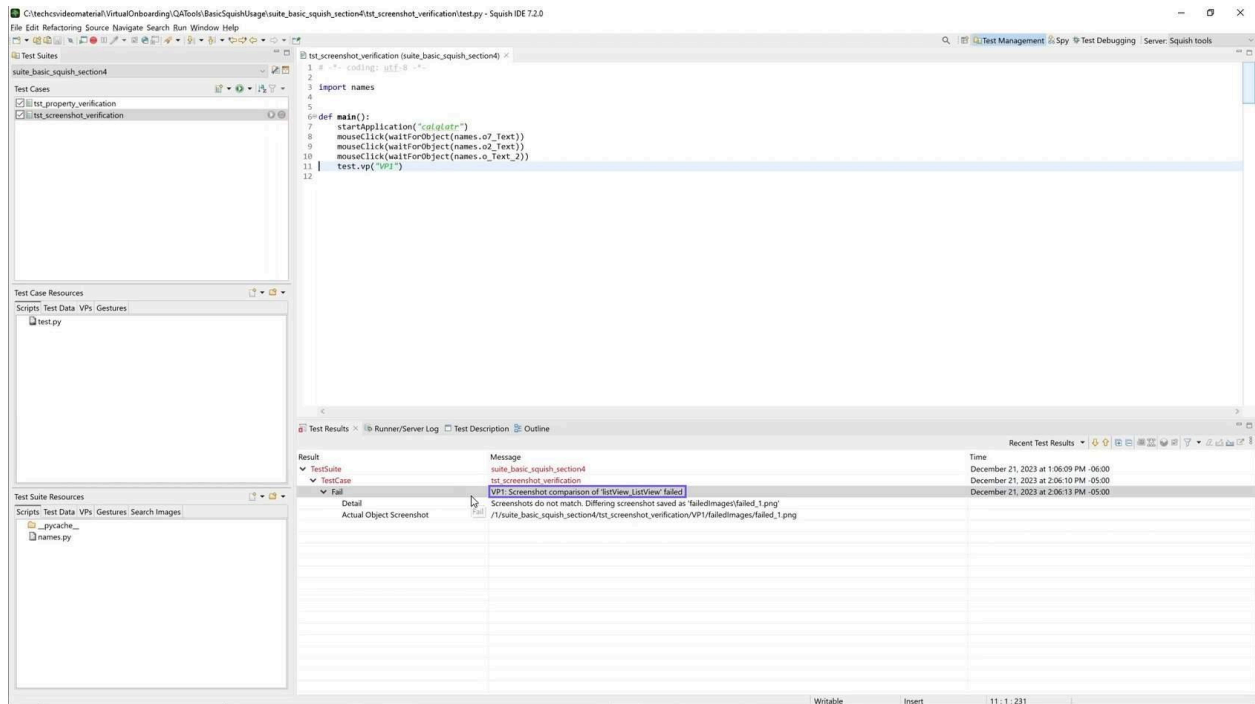
Use the run button to execute the test and observe the results in the Test Results View. Expand the details of the Pass in the Test Results View to see the verification point comparison.

08 Modify the script



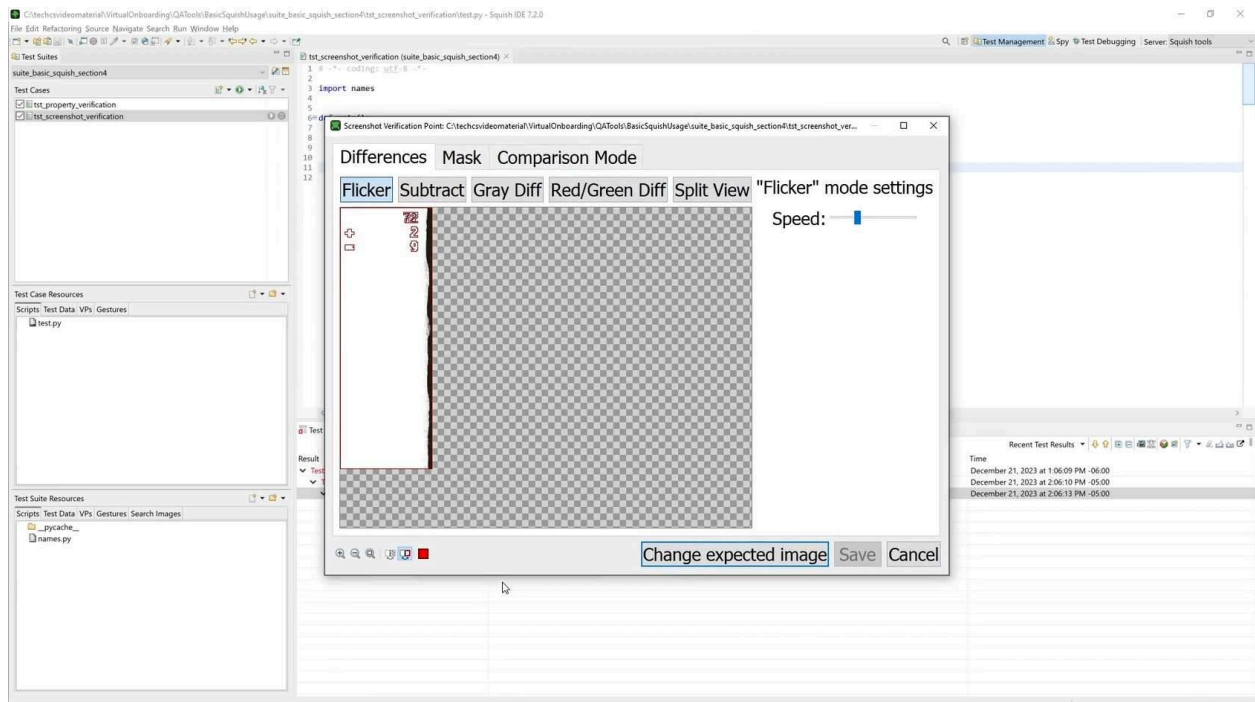
Modify the script to create a different calculator result by removing one of the mouse click functions.

09 Testing a failure



Run the test to observe a failure. Expand the details of the Pass in the Test Results View to see the verification point comparison.

10 Viewing Differences

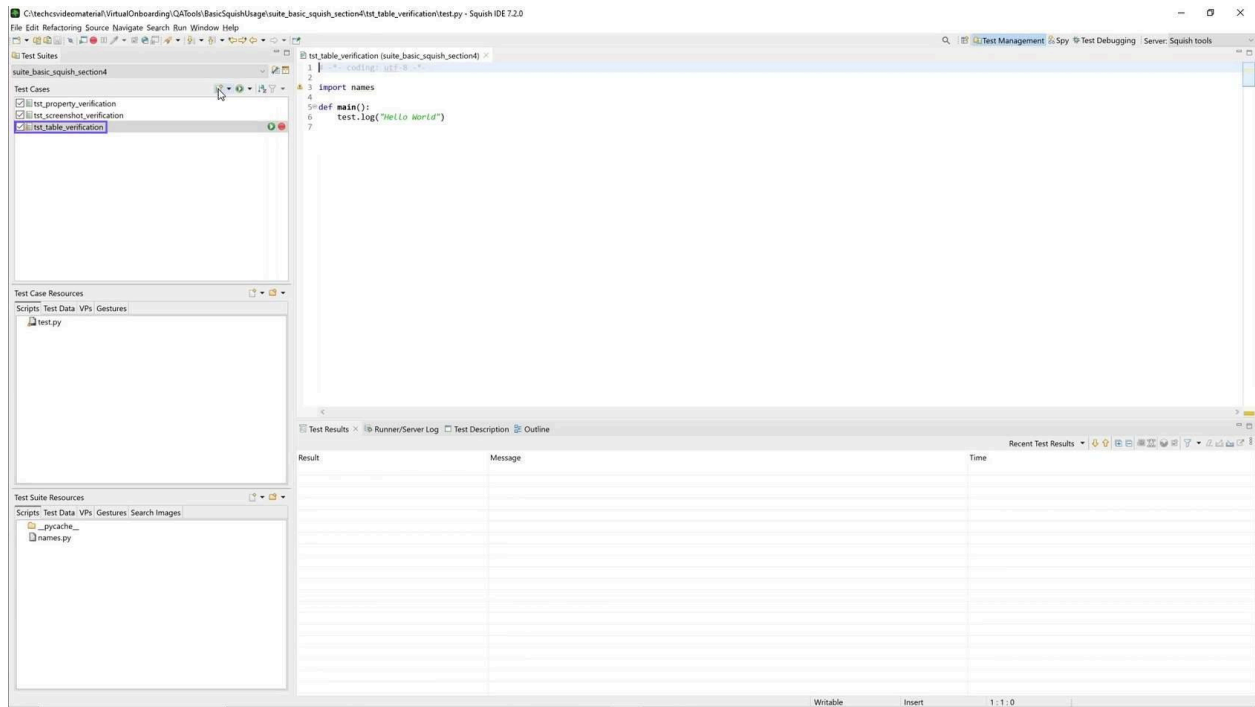


After it finishes, Right-click the failed screenshot verification and select "View Differences" to analyze the discrepancies.

Table Verification

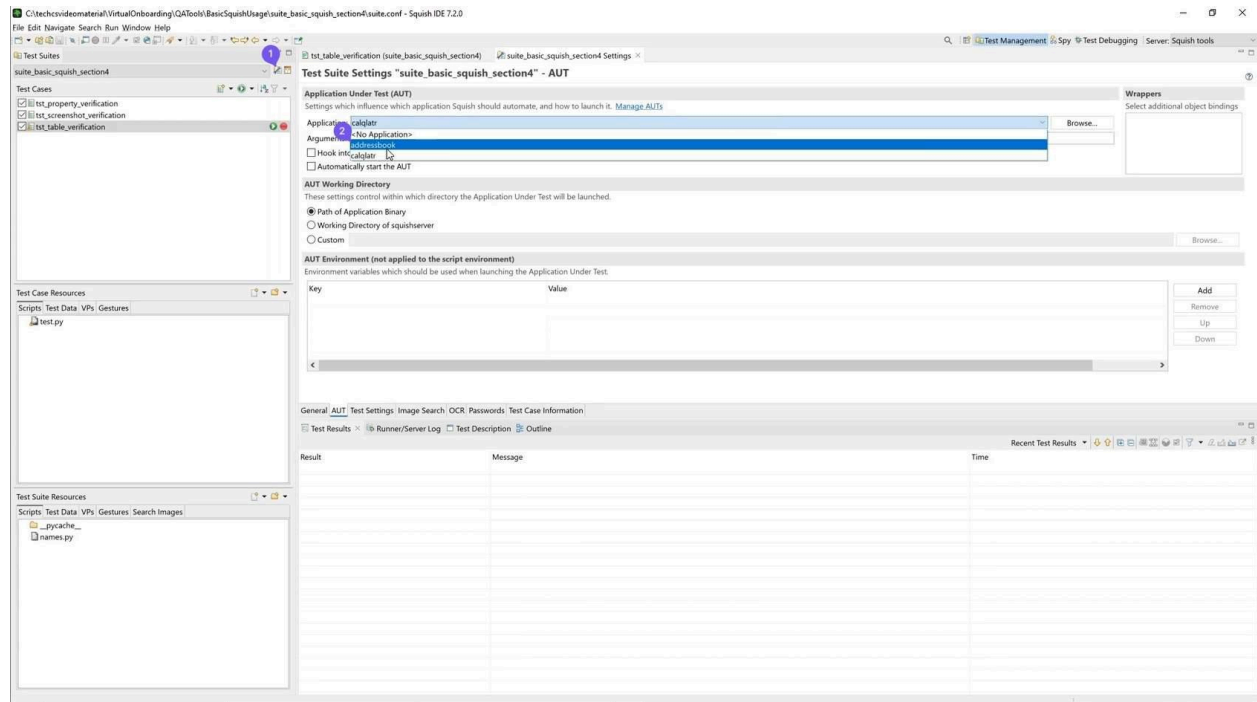
STEPS - Creating a New Table Verification

01 Create a new test case



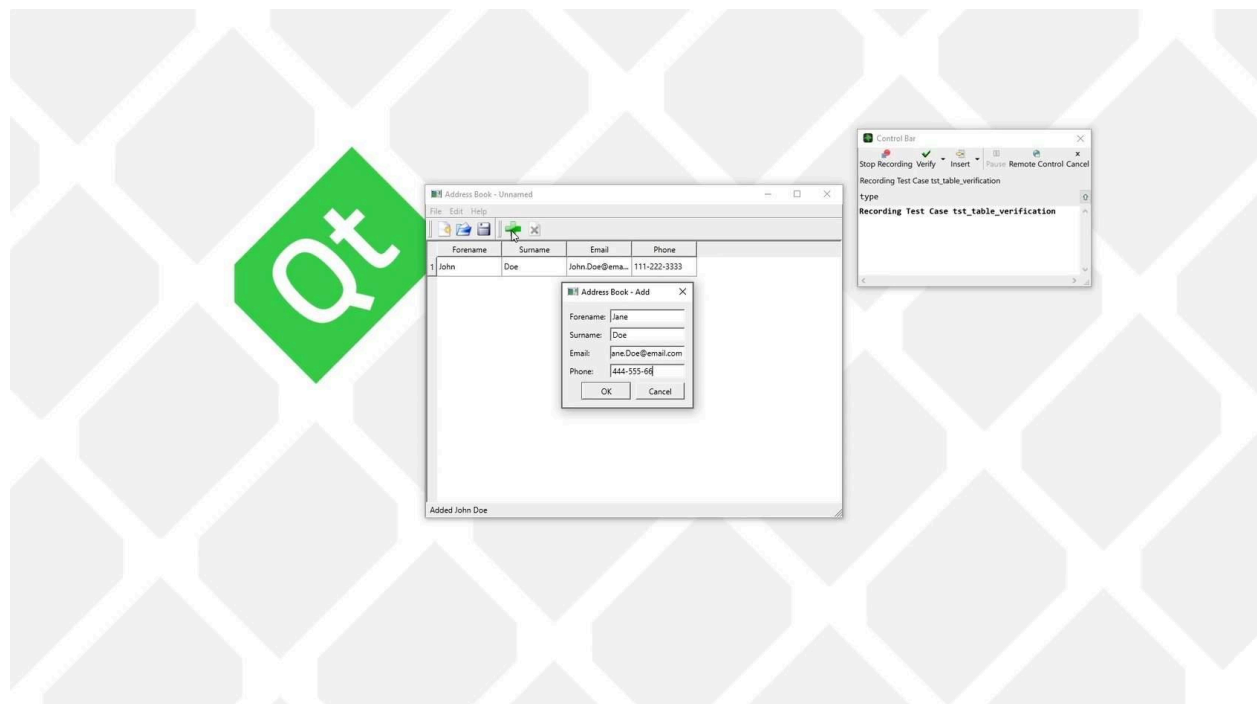
Begin by adding a new test case, “tst_table_verification”.

02 Set the AUT



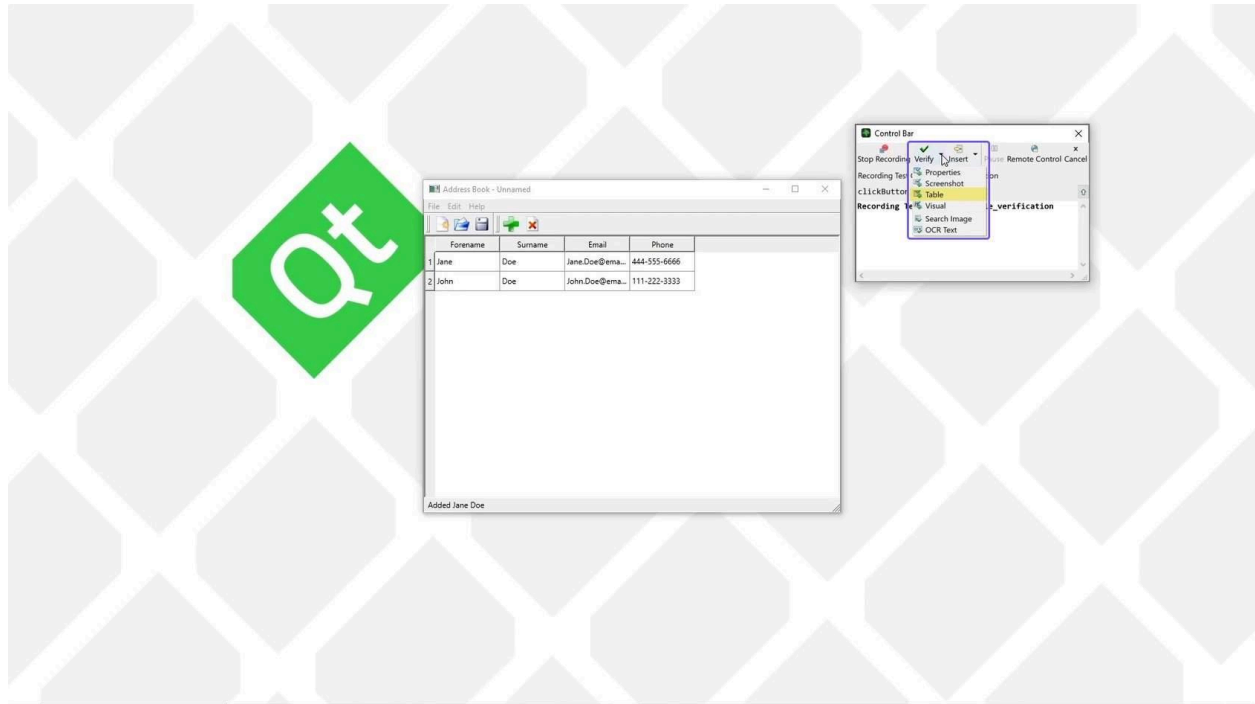
Use the Test Suite Settings icon to access the AUT tab and set the address book example as the default AUT.

03 Start recording a text, add entries to the address book



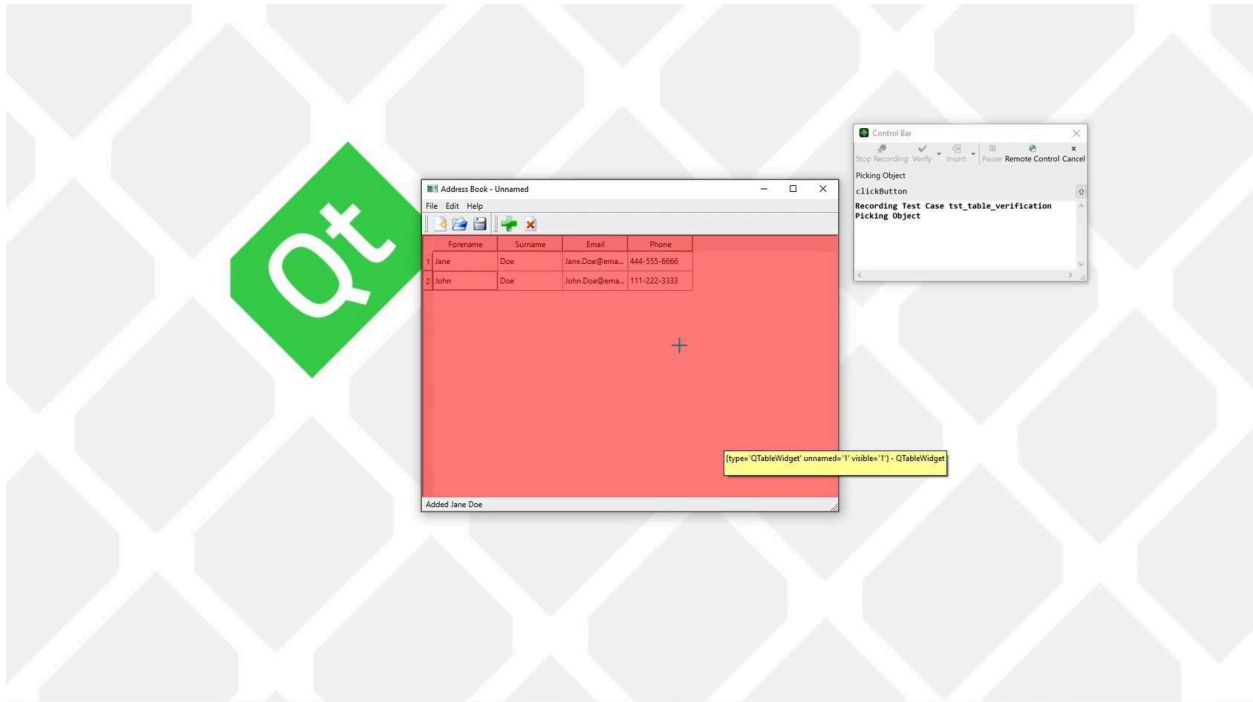
Start recording the test by clicking the record button. Add entries to a new address book to populate the table.

04 Select table verification



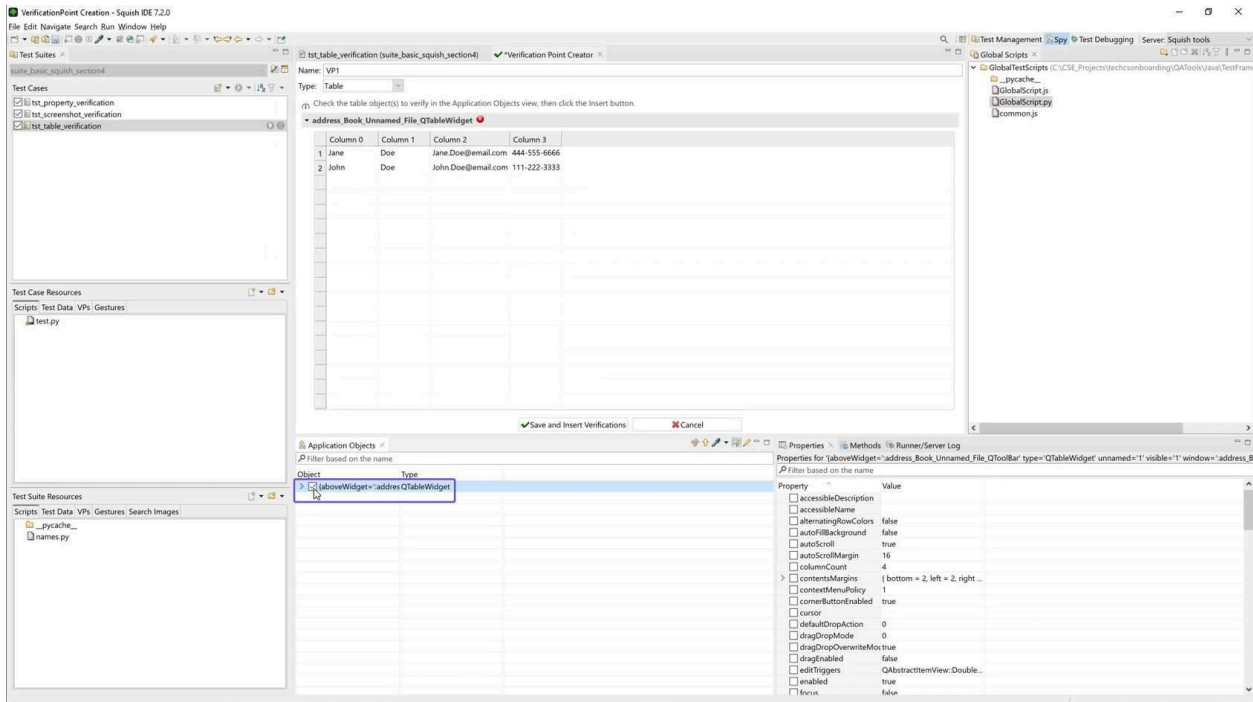
In the Control Bar, select table verification from the dropdown next to the Verify button. Select the picker tool when returned to the Squish IDE.

05 Select the table with the picker tool



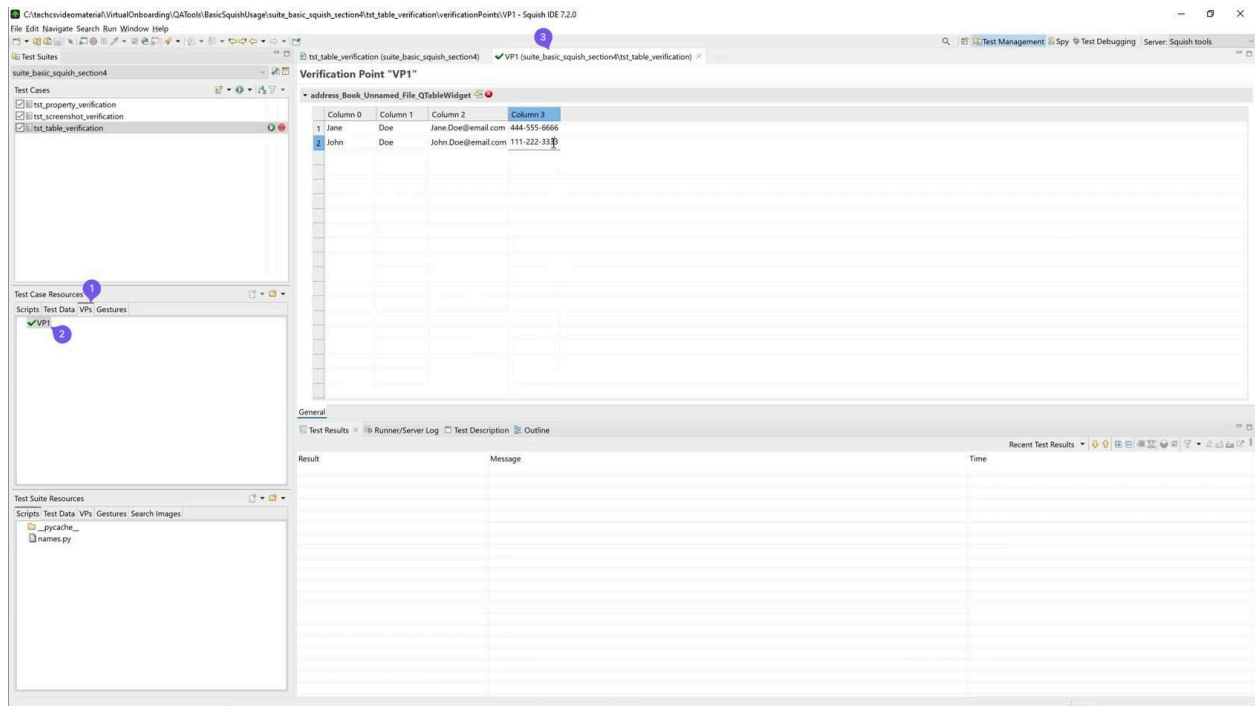
Use the Picker tool to select the QWidget type table in the address book for verification.

06 Choose the object from the Application Object view



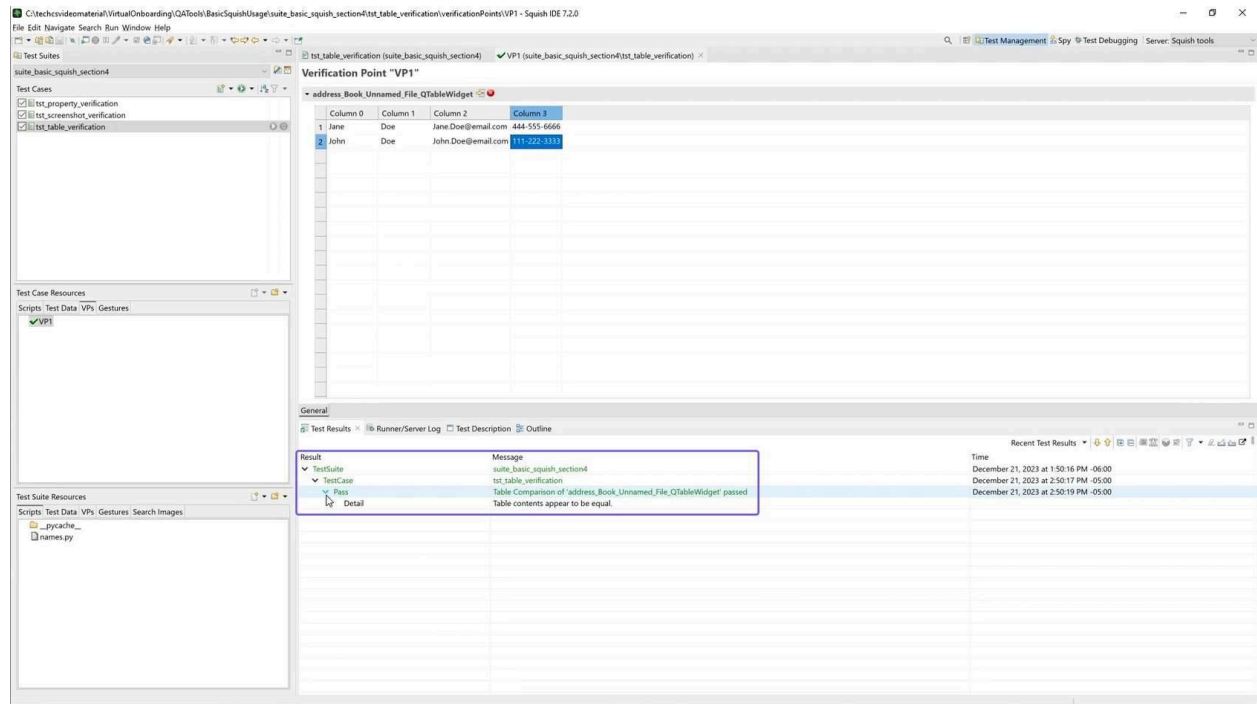
Select the QTableWidgetItem from the Application Object view to view the table in the Verification Point Creator view and set up the table verification. Save and insert the verification and stop the recording.

07 Explore the verification point editor



Click the VPs tab to open the Verification Point Creator view and set up the table verification. Here you can modify the the VP if necessary. For this example, we do not modify anything.

08 Run and verify



Run the test to observe the entries added to the address book and check the results in the Test Results view. Expand the details of the Pass in the Test Results View to see the comparison of the table value with the expected value.

Visual Verification

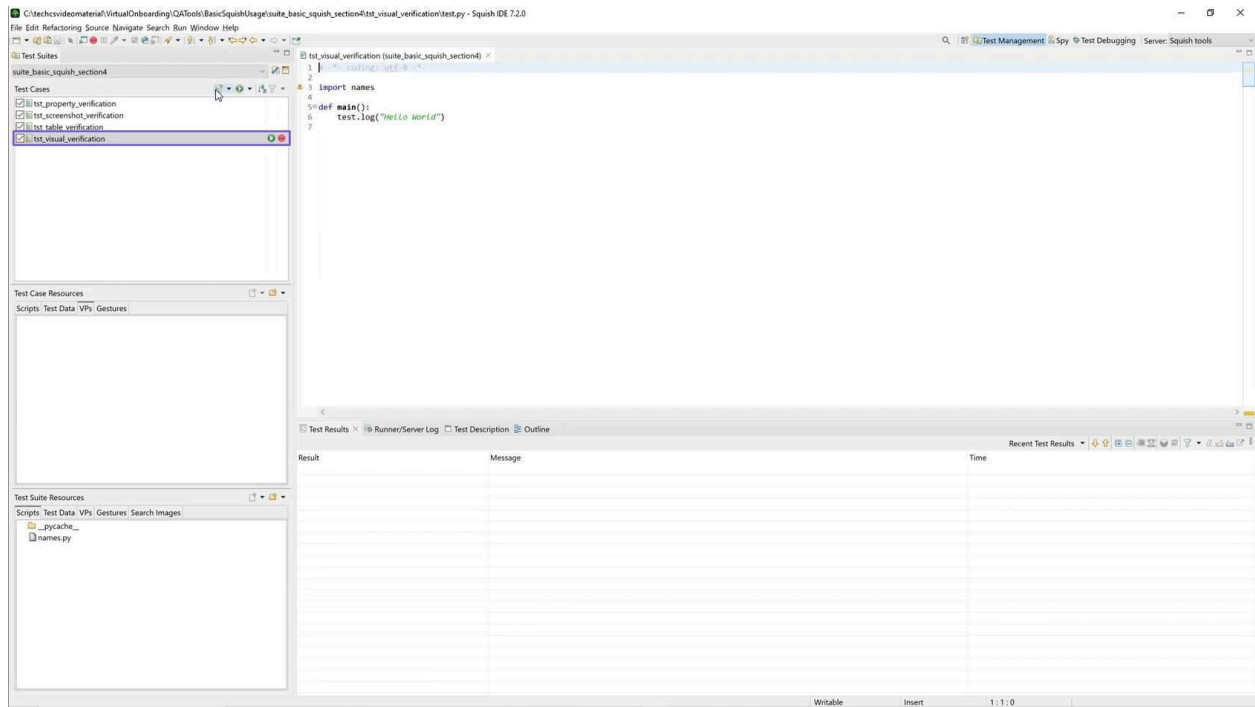
Visual Verification verifies GUI by selecting a visual section of the UI and validates any or all of the following:

- Content Properties
 - Properties which influence the way a GUI object is rendered on the screen, including textual content, numerical value, or a current state. These properties are used as part of the Identity Check and provide a way to differentiate elements from one another.
- Geometric Properties
 - Properties that describe an object's geometry such as x and y position, width, and height.
- Screenshot
 - A pixel-by-pixel comparison of an individual GUI object, group of objects, or an entire window.

Visual comparisons also utilize the **test.vp()** function and can be used in test scripts accordingly.

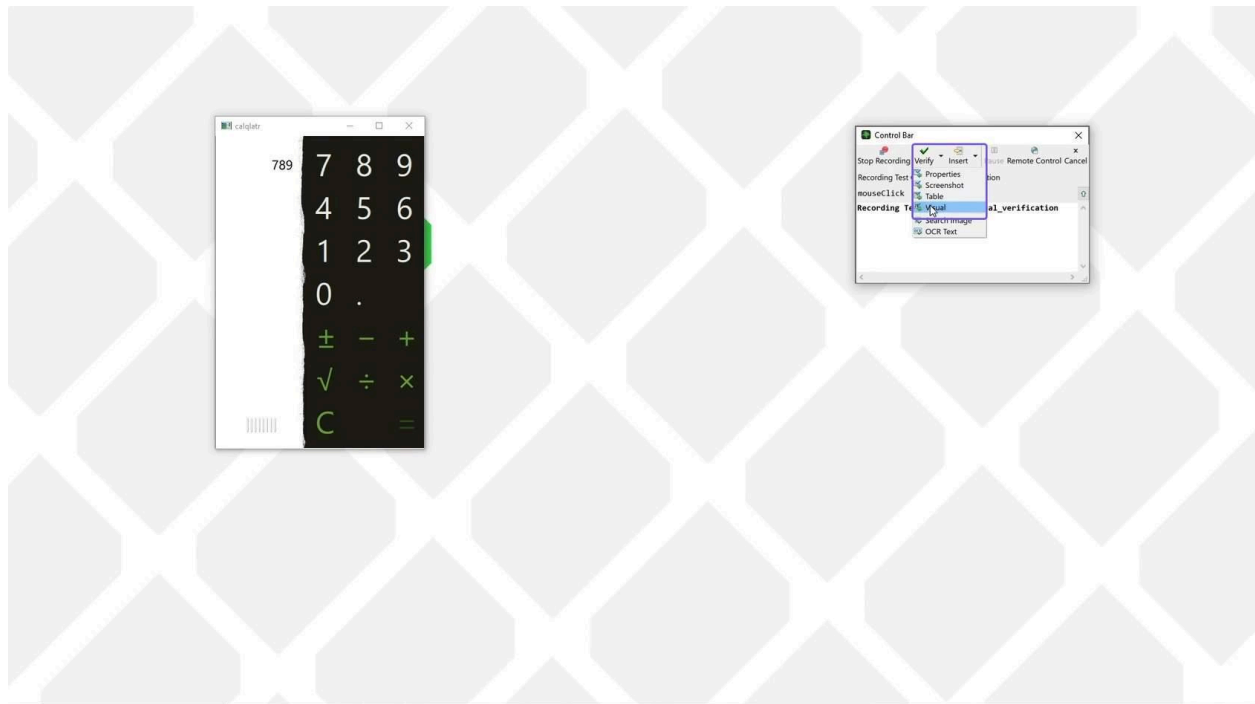
STEPS - How to Create a Visual Verification

01 Create new test case and set calculator as AUT



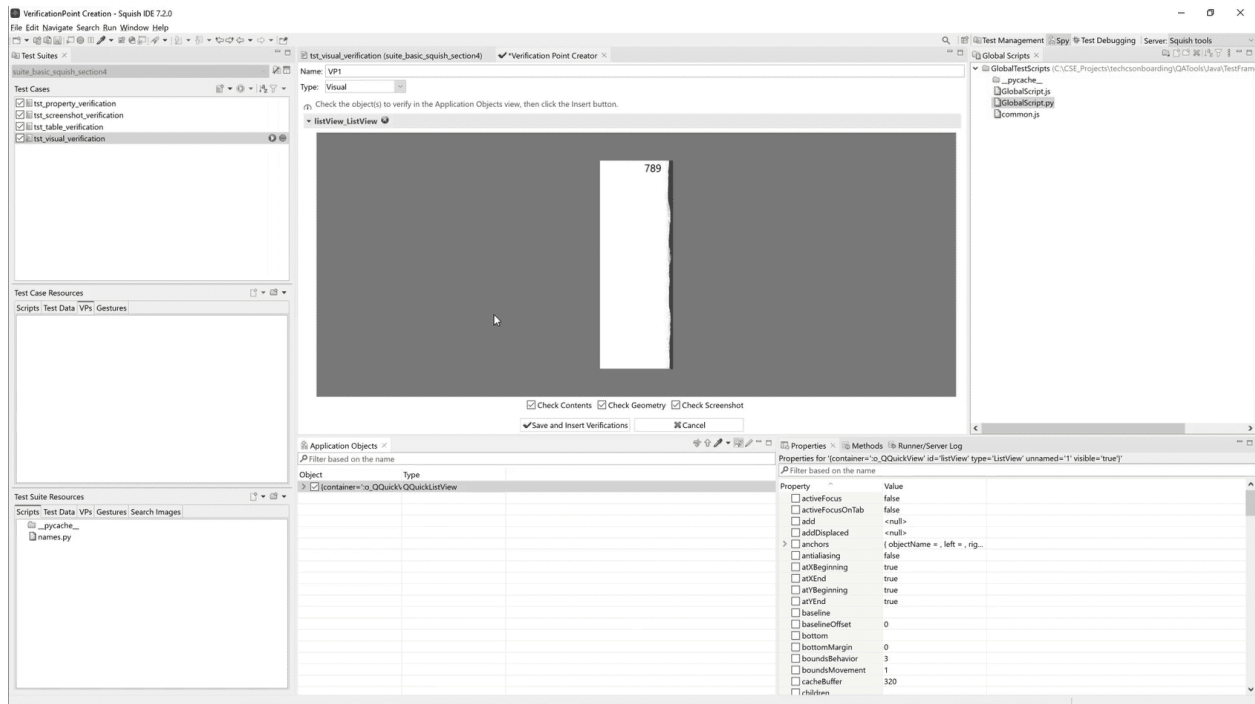
Add a new test case, "tst_visual_verification" using the calculator example as the AUT in the test suite.

02 Initiate test recording and select object verification



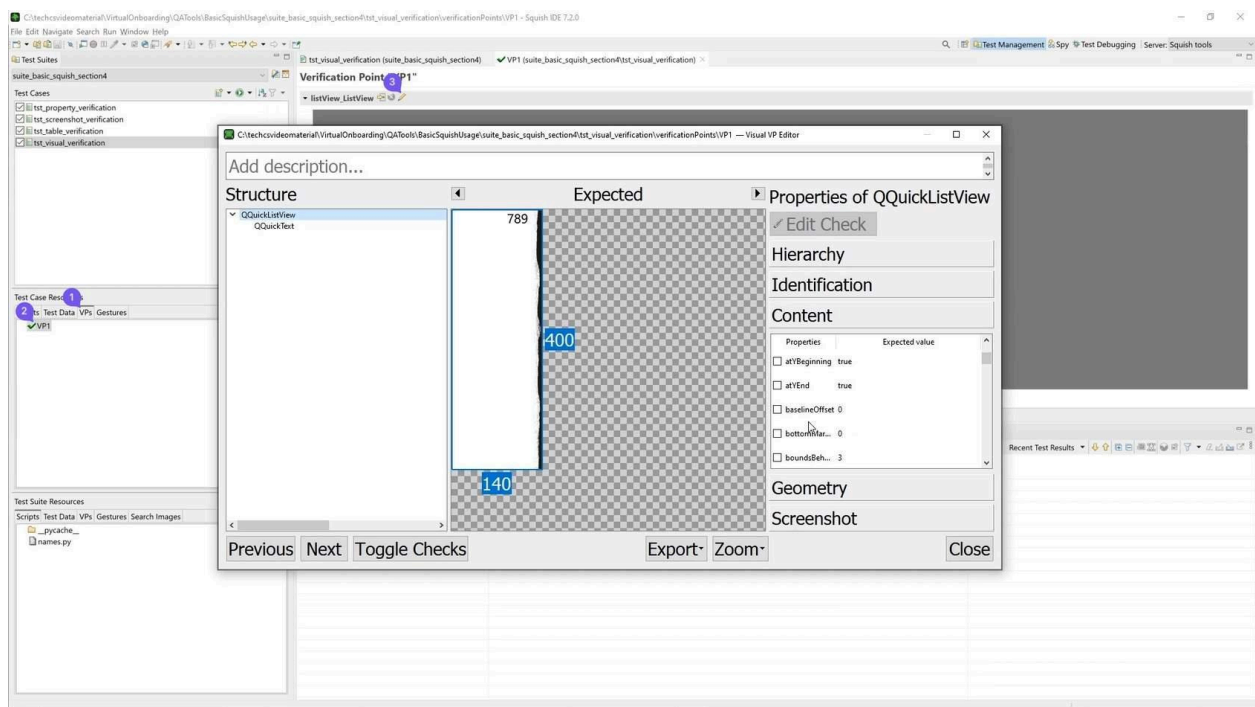
Start recording the test by clicking the record button and input a simple operation in the calculator. In the Control Bar, select visual verification from the dropdown next to the Verify button.

03 Select the calculator result area and open in the Verification Point Creator



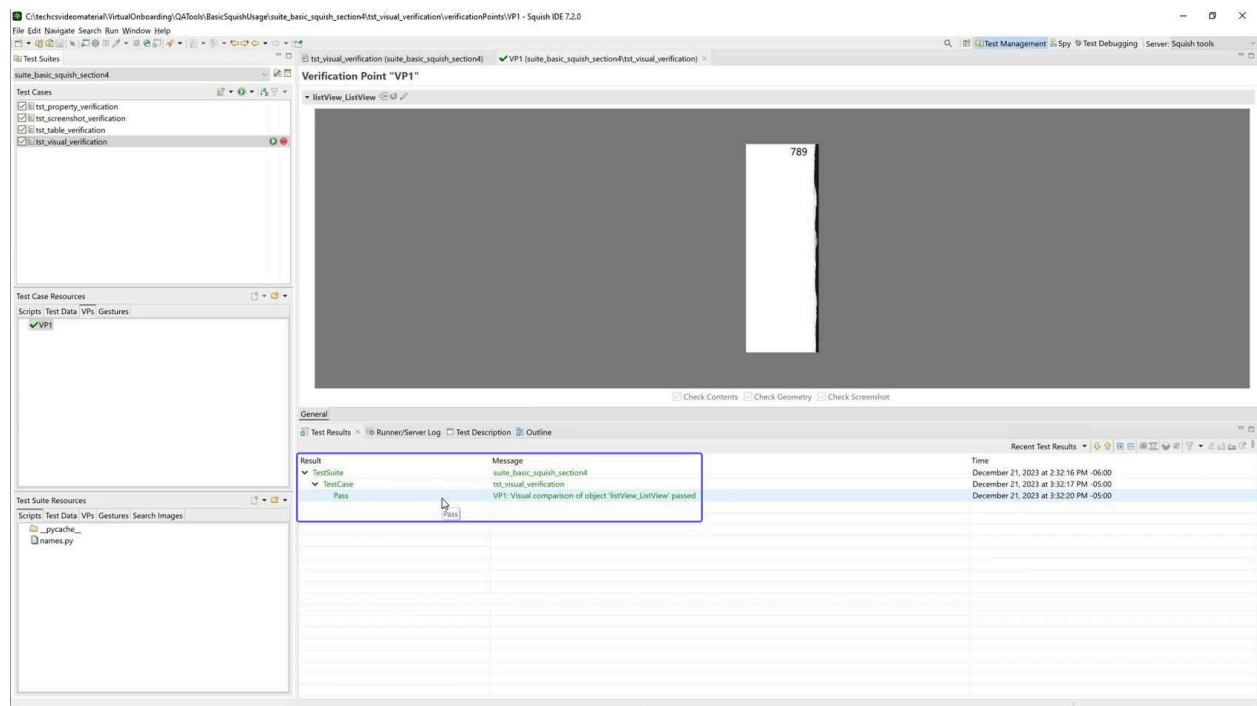
Use the Picker tool to select the QQuickListView object displaying the calculator results. Once selected, view the object in the Verification Point Creator view and set up the visual verification. Save the verification and stop the recording.

04 Explore the Visual Verification Point Editor



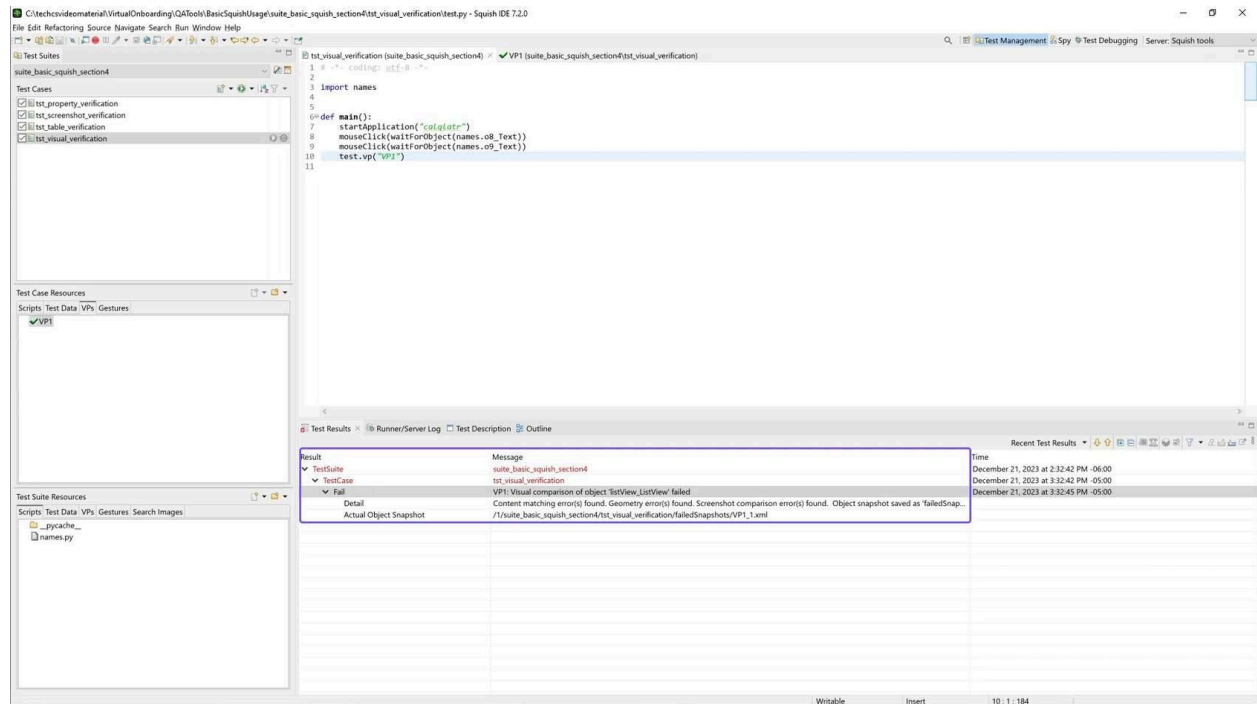
Navigate to the Verification Point Editor to explore how what can be edited with visual verifications. Here, we can add or remove properties used in content checks, edit geometry checks, verify the sizing and position of the object, and screenshot checks. Properties can be added or removed.

05 Run and verify



Run the test to observe the digits added to the calculator and check the results in the Test Results view. The test passed!

06 Testing a failure



Modify the python script by removing a mouseClicked function to observe a failure as we run the test once more.

Search Image Verification

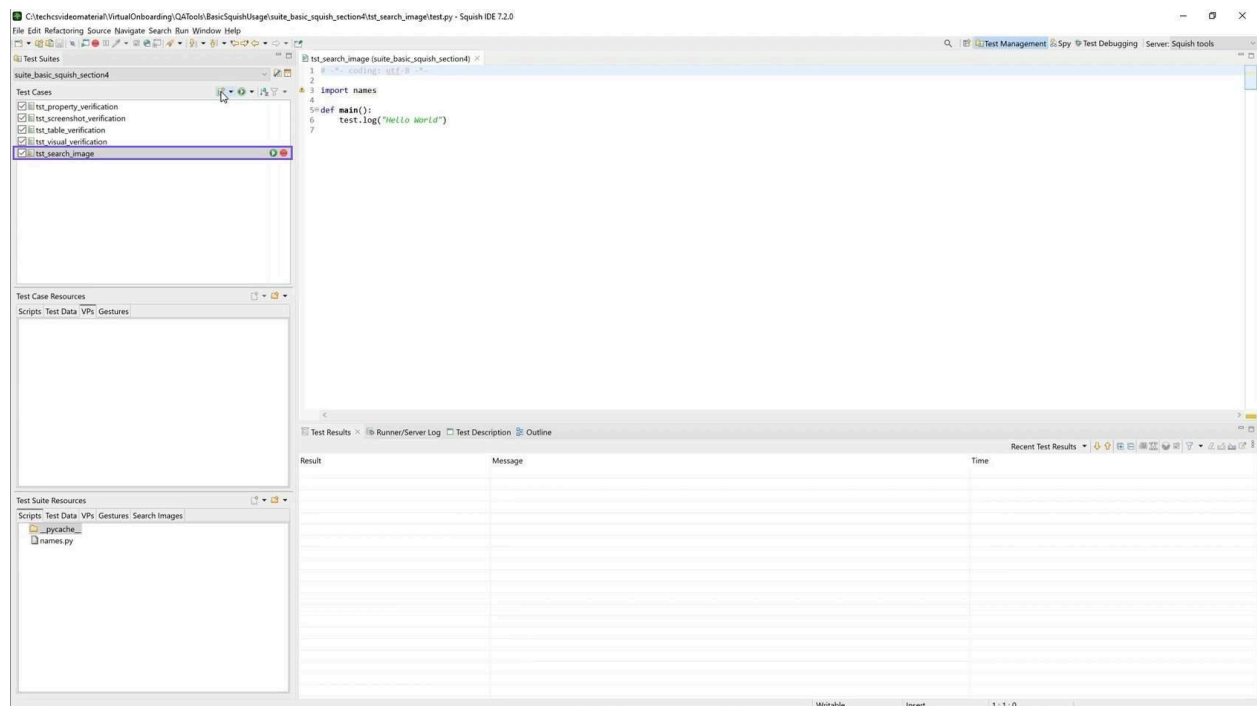
Search Image Verification verifies an image is present somewhere in the window where the AUT is running. A search image is created by first capturing a snippet of the AUT or desktop. The image can then be used within the [test.imagePresent\(\)](#) function to perform a search for it within the AUT. Search images are saved as resources within a test case or test suite and can be viewed within the Squish IDE, like verification points.

Python

```
test.imagePresent("image")
```

STEPS - Creating New Search Image Verification

01 Create new test case with the calculator as the AUT



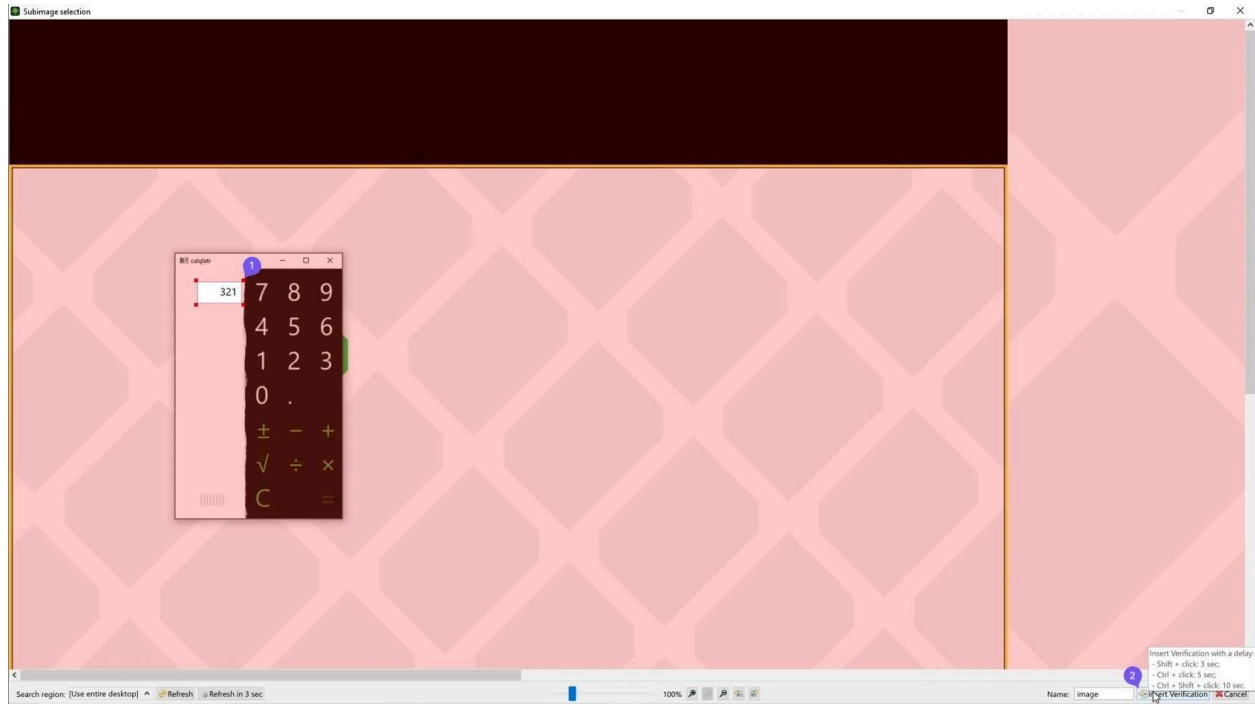
Add a new test case named “tst_search_image” using the calculator example as the AUT in the test suite.

02 Initiate test recording and select Search Image



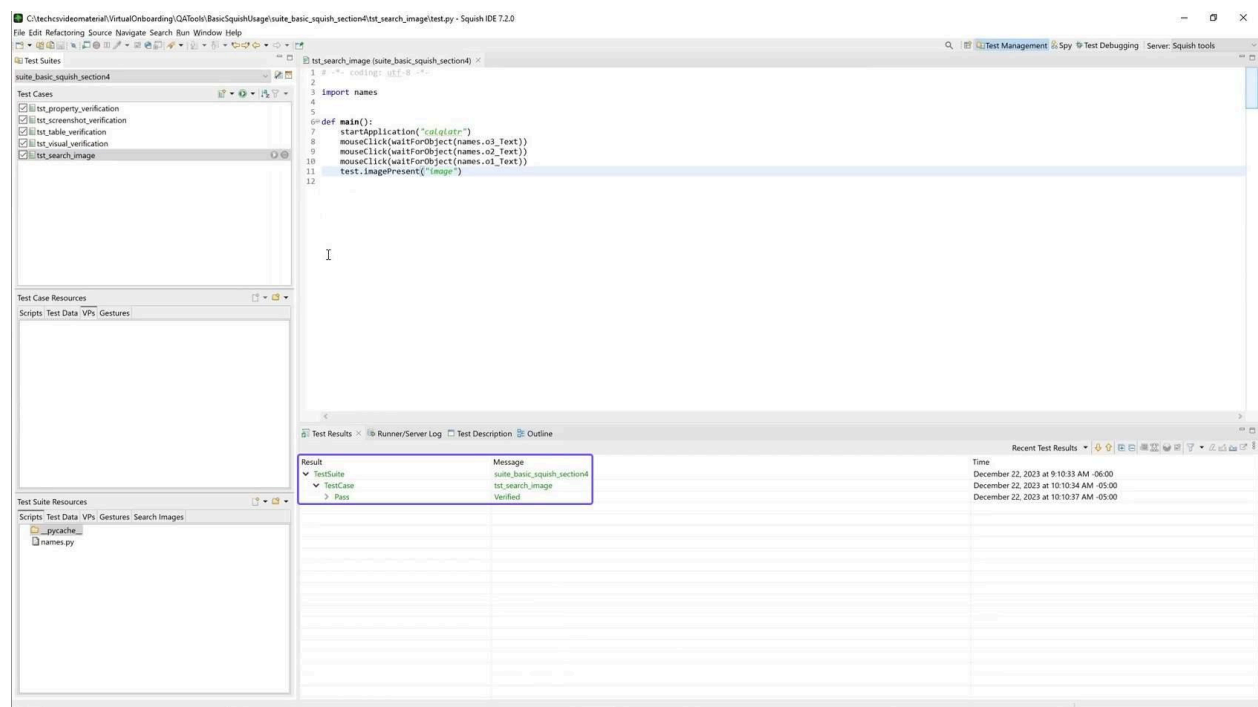
Start recording the test and input some values in the calculator. Use the Verify dropdown in the Control Bar to select Search Image verification. The Subimage selection window will appear.

03 Create a snippet with Subimage selection window



Select a snippet from the calculator result to be saved as a search image by clicking and dragging a rectangle over the input values. Click "Insert Verification". Stop the recording.

04 Run and Verify



Run the test to play back the input and test the verification.

Optical Character Resolution (OCR) Text Verification

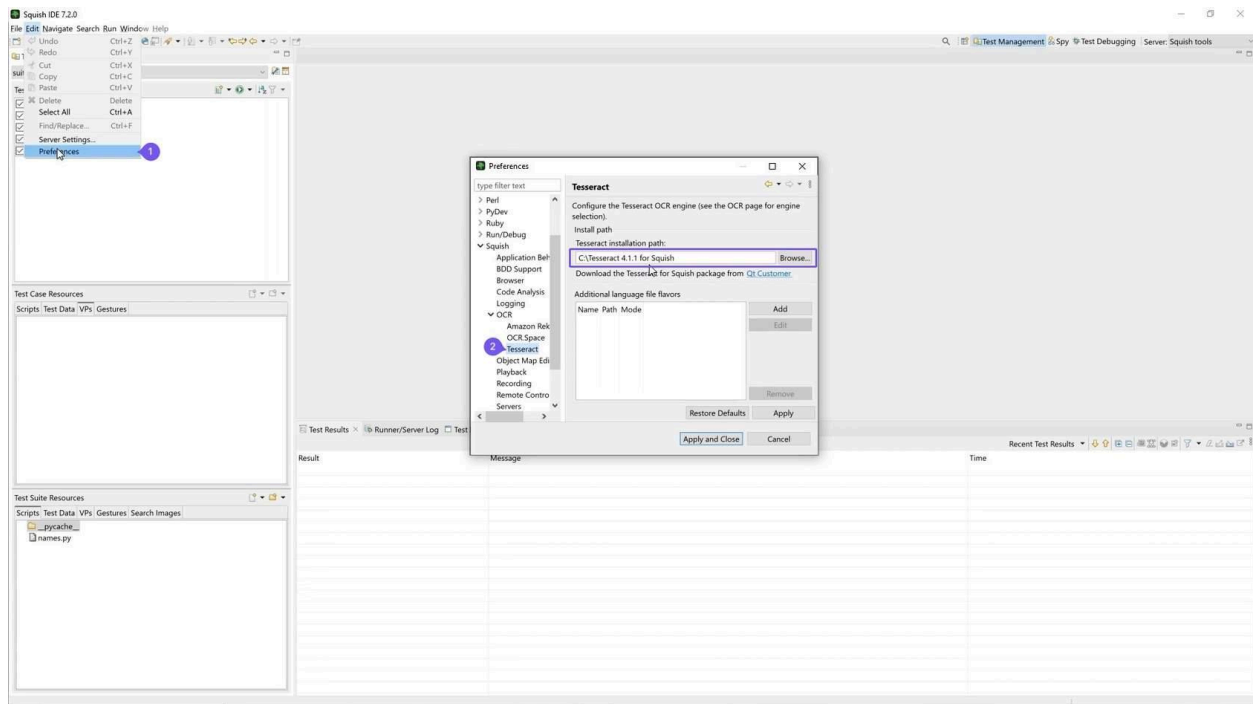
Optical Character Resolution (OCR) Text verifies that some text is present within a defined region. Squish uses the free Tesseract OCR library as its primary engine to analyze pixel image data and recognize text in it. Occurrences of that text can then be detected in the GUI using [test.ocrTextPresent\(\)](#) function. **Tesseract for Squish** can be downloaded from the [Downloads](#) page in the [Qt Customer Portal](#).

Python

```
test.ocrTextPresent("calqlatr");
```

STEPS - How to Create an OCR Text Verification

01 Check OCR library is installed and linked



Ensure tesseract is installed. Navigate to **Edit > Preferences > Squish > OCR > Tesseract** to set the correct path to the Tesseract engine.

02 Record a new test case and select OCR verification



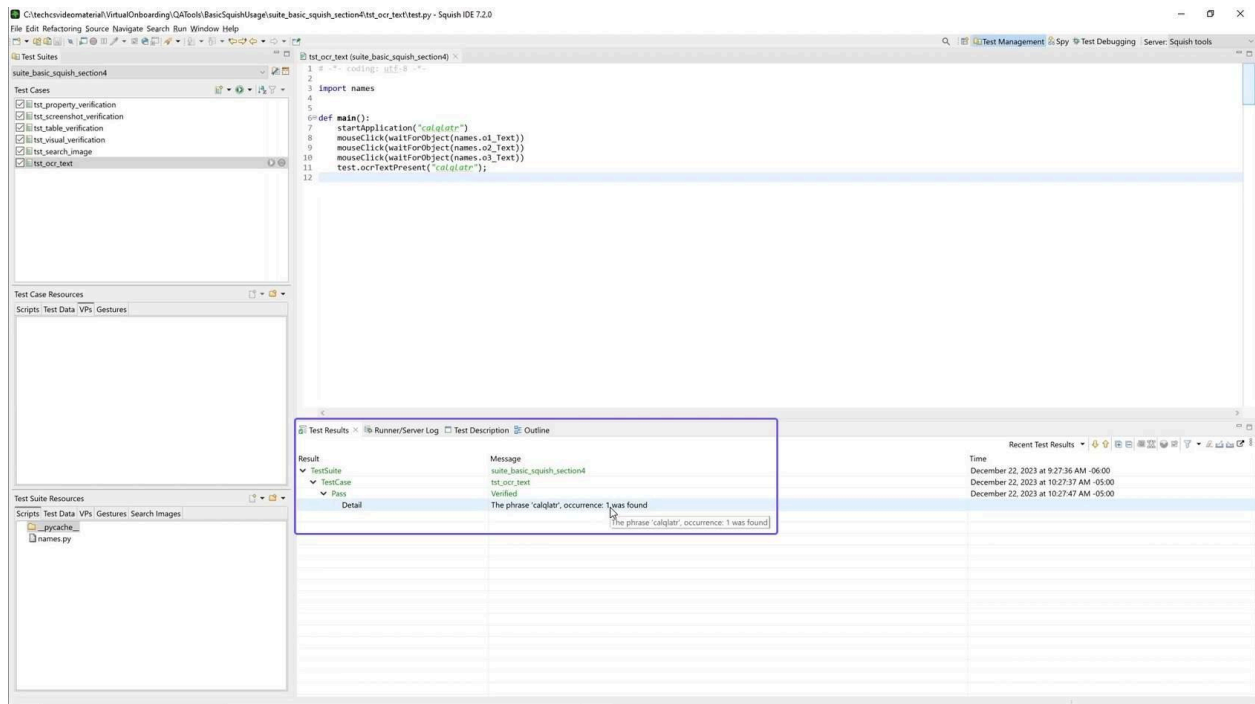
Create a new test case named “tst_ocr_text” using the calculator example. Record some input and then select OCR text verification from the Verify dropdown. The OCR verification window will appear.

03 Select area for OCR verification and insert



Identify and select the desired text in the OCR verification window. Verify the chosen text and select "Insert Verification" to confirm. Stop the recording.

04 Run and verify



Run the test, Expand the details of the Pass in the Test Results View to see the OCR text verification point.

Summary

Setting Up Test Suites and Cases

A test suite acts as a container for multiple test cases, each focusing on specific aspects of the application under test (AUT). Start by adding a new test case to the suite, giving it a descriptive name following conventions like “tst_” prefix for clarity. Configure the default AUT for the test suite under **Edit > Server Settings > Manage AUTs**, ensuring that the environment is prepared for recording and verifying tests.

Recording and Verification Process

The recording feature in Squish IDE helps in automating the process of creating test scripts. By interacting with the AUT while the recording is active, Squish captures these interactions and automatically generates the corresponding test script code. This approach simplifies test creation. Following the recording, various verification methods like OCR text, visual, table, and search image verifications are used to validate different aspects of the AUT, ensuring comprehensive testing.

Utilizing Different Verification Types

Squish IDE offers a range of verification types to suit different testing needs. This includes OCR text verification (requiring an external OCR engine like Tesseract), visual verification (comparing GUI elements against expected appearances), table verification (ensuring table contents match expectations), and search image verification (detecting occurrences of specific images). Each verification type addresses different testing criteria, from text presence to visual accuracy, enhancing the robustness of the tests

Test Execution and Analysis

Once a test case is recorded and verifications are set up, executing the test is straightforward with the run button. Squish executes the test script, applying all the verification points. The Test Results view in Squish IDE provides detailed outcomes, indicating whether each test passed or failed. For failed tests, analyze the results to understand discrepancies, making it an invaluable tool for debugging and refining tests.